Projekti tüüpvead

Sisukord

1 Vead dokumendis....3
1.1 Viga 1 (liiga vähe olemitüüpe)....3
1.2 Viga 2 (ebasobiv funktsionaalne allsüsteem)....3
1.3 Viga 3 (liiga vähe töökohti)....3
1.4 Viga 4 (liiga üldine pädevusala – Töötaja)....4
1.5 Viga 5 (liiga pinnapealne pädevusalade valik)....5
1.6 Viga 6 (puudub omanik)....5
1.7 Viga 7 (funktsionaalne allsüsteem=roll)....6
1.8 Viga 8 (viitamise puudumine)....6
1.9 Viga 9 (allsüsteemide halvad nimed)....7
1.10 Viga 10 (registrite ebasobiv skoop)....7
1.11 Viga 11 (puudulik süsteemianalüüs)....7
1.12 Viga 12 (puuduv juhtimistasand)....8
1.13 Viga 13 (valesti valitud aruanded)....9
1.14 Viga 14 (seaduste mitte tundmine)....9
1.15 Viga 15 (mudeli halb kvaliteet)....10
1.16 Viga 16 (surrogaatvõtmed kontseptuaalses andmemudelis)....12
1.17 Viga 17 (välisvõtmed kontseptuaalses andmemudelis)....13
1.18 Viga 18 (puuduvad üldistuste hulgad)....14
1.19 Viga 19 (kasutusjuhtude vale jaotus)....16
1.20 Viga 20 (kasutusjuhtude vale skoop)....16
1.21 Viga 21 (kasutusjuhtude halvad nimed)....17
1.22 Viga 22 (puuduvad viited andmebaasioperatsioonidele)....17
1.23 Viga 23 (ebatäpsed andmebaasioperatsioonide järeltingimused)....18
1.24 Viga 24 (andmebaasioperatsioonide ebasobiv skoop)....19
1.25 Viga 25 (vead seisundidiagrammis)....19
1.26 Viga 26 (ei saa genereerida arveid)....24
2 Vead SQL-andmebaasis....25
2.1 Viga 27 (puuduvad unikaalsuse kitsendused)....25
2.2 Viga 28 (puuduvad välisvõtme kitsendused)....28
2.3 Viga 29 (puuduvad CHECK kitsendused)....30
2.4 Viga 30 (klassifikaatori koode genereerib andmebaasisüsteem)....31
2.5 Viga 31 (halvasti nimetatud võtmeveerud)....33
2.6 Viga 32 (tühjade stringide lubamine tekstiväljas)....36
2.7 Viga 33 (puuduvad veateated)....37
2.8 Viga 34 (ebatäpsed veateated)....39
2.9 Viga 35 (ebareaalsed ajahetked – puuduvad CHECK kitsendused)....40
2.10 Viga 36 (CHECK kitsendused, mis ei arvesta aja kulgemisega)....41
2.11 Viga 37 (puuduvad vaikimisi väärtused)....42
2.12 Viga 38 (ebakorrektsed vaikimisi väärtused)....43
2.13 Viga 39 (unikaalsuse kitsenduste asjatu dubleerimine)....44
2.14 Viga 40 (üldine klassifikaatorite tabel)....45
2.15 Viga 41 (kitsendustele annab nime andmebaasisüsteem)....46

2.16 Viga 42 (ebasobivad kompenseerivad tegevused)....48
2.17 Viga 43 (CHECK kitsenduste dubleerimine)....49
3 Vead kasutajaliideses....50
3.1 Viga 44 (seotud andmeid ei saa valida)....50
3.2 Viga 45 (olemid pole eristatavad)....52
3.3 Viga 46 (valiku tegemisel korduvad andmed)....55
3.4 Viga 47 (valiku tegemisel sorteerimata andmed)....57
3.5 Viga 48 (kasutaja näeb surrogaatvõtmete väärtuseid)....60
3.6 Viga 49 (väljad ei eristu piisavalt)....61
3.7 Viga 50 (ebasobiva suurusega väljad)....62
3.8 Viga 51 (seisundi valimine)....63
3.9 Viga 52 (halvad veateated)....64
3.10 Viga 53 (ebajärjekindel formaatimine)....66
3.11 Viga 54 (ebaselge andmete tähendus)....67
3.12 Viga 55 (ebapiisav info piirangute kohta)....69
4 Soovitused....70
4.1 Soovitus 1 (äri tundmaõppimine)....70
4.2 Soovitus 2 (pakettide kasutamine)....71
Kasutatud materjalid....73

Järgnevalt selgitatakse tüüpilisi vigu, mis on projektide ülevaatamisel silma hakanud.

1 Vead dokumendis

Järgnevalt kirjeldatakse vigu süsteemiarenduse mudelites. Mudelid võivad olla nii visuaalsed (diagrammid e skeemid), tekstilised kui ka nende kombinatsioon.

1.1Viga 1 (liiga vähe olemitüüpe)

Kontseptuaalses andmemudelis on olemitüüpe liiga vähe.

Kommentaar: Olemitüüpe peab "Andmebaasid I" projekti kontseptuaalses andmemudelis olema vähemalt seitse.

1.2Viga 2 (ebasobiv funktsionaalne allsüsteem)

Detailseks käsitlemiseks pole valitud organisatsiooni põhitegevusele vastavat funktsionaalsete allsüsteemi.

Kommentaar: Projektis tuleb detailseks analüüsimiseks valida organisatsiooni ühele põhitegevusele vastav allsüsteem. Näiteks, kui valite üldiseks teemaks kaupluse infosüsteemi, siis tuleb täpsemaks käsitlemiseks valida kaupade funktsionaalne allsüsteem, müükide funktsionaalne allsüsteem või tellimuste funktsionaalne allsüsteem, aga mitte näiteks töötajate funktsionaalne allsüsteem või klassifikaatorite funktsionaalne allsüsteem. Näiteks kaks viimati nimetatud allsüsteemi on osaks paljudest erineva valdkonna infosüsteemidest ja neid võib kaupluse infosüsteemi kontekstis nimetada administratiivseteks (tugiteenuseid pakkuvateks) allsüsteemideks.

1.3Viga 3 (liiga vähe töökohti)

Valitud funktsionaalsel allsüsteemil ei ole vähemalt kahte loogilist töökohta.

Kommentaar: Funktsionaalne allsüsteem pakub pädevusalade esindajatele teenuseid. Erinevate pädevusalade esindajatel on erinevad ülesanded, mida nad peavad infosüsteemi abil täitma. Funktsionaalsetest allsüsteemidest kujuneb välja andmebaasi kasutavate programmide komplekt. Loogiline töökoht tähendab seda, et programmidele luuakse kasutajaliides, mille kaudu saab töökoha kasutaja (pädevusala esindaja) oma ülesandeid täita. Näiteks kaupade funktsionaalsel allsüsteemil on üks loogiline töökoht kaupade halduril, kes saab kaupade andmeid lisada ja muuta ning teine loogiline töökoht kliendil, kes saab aktiivsete kaupade andmeid vaadata.

1.4Viga 4 (liiga üldine pädevusala – Töötaja)

Pädevusalaks on "Töötaja pädevusala" ja detailsemad pädevusalad (vastavad erinevatele ametikohtadele või rollidele) puuduvad.

Kommentaar: "Töötaja" pädevusala on teiste pädevusalad hulgas mõtet kirjeldada siis, kui kõigil organisatsiooni töötajatel leidub ühesuguseid ülesandeid, mida nad peavad "töötaja" rollis täitma. Enamasti on erinevatel töötajatel erinevad ülesanded ning "töötaja" pädevusala kirjeldus oleks seega liiga üldine. Näiteks ülikooli töötajad on nii rektor kui ka koristaja, nende mõlema töö on väga oluline, kuid töö ülesanded ja vastutuse ulatus on väga erinev.

Süsteemianalüüsi käigus tuleb leida infosüsteemi kasutajate *rollid.* Üks võimalus *esialgse* rollide nimekirja leidmiseks on käsitleda organisatsiooni hetke ametikohti rollidena. See on hea esialgne lähendus, kui süsteemianalüütik ei tunne piisavalt modelleeritavat valdkonda. Raamatukogu ***töötajate ametikohad*** on Tallinna Tehnikaülikooli raamatukogu põhjal näiteks:

- direktor,
- asedirektor,
- osakonnajuhataja,
- raamatukoguhoidja,
- bibliograaf,
- süstematiseerija,
- kataloogija,
- keeletoimetaja,
- ainespetsialist,
- projektijuht,
- näituste kuraator.

Töötajate ametikohtadele või rollidele vastavad pädevusalad on *organisatsiooni sisesed* pädevusalad.

Mõnikord on mõistlik üldistada töötajate ametikohtadele vastavaid rolle üldisemateks rollideks, sest mõned tööülesanded on seotud kõigi üldistatavate rollidega. Näiteks õppeinfosüsteemis on mõistlik üldistada ametikohtadele vastavad rollid:

- professor
  - täisprofessor
  - kaasprofessor
  - abiprofessor
- dotsent
- lektor
  - vanemlektor
  - lektor
- assistent
- õpetaja

rolliks *Õppejõud*. Osa ülesandeid on ühised kõigile õppejõududele, kuid osa ülesandeid on spetsiifilised ametikohale vastavale rollile.

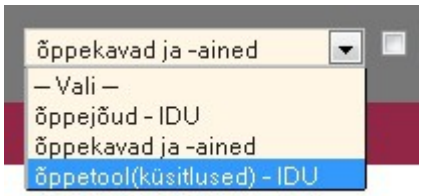
Tallinna Tehnikaülikooli õppeinfosüsteemis jaguneb õppejõu roll omakorda:

- põhiõppejõud,
- asendusõppejõud,
- kaasõppejõud,
- abiõppejõud.

Organisatsiooni väliste pädevusalade leidmiseks tuleb mõelda *kliendi (sõltuja) /teenusepakkuja* suhetele, milles organisatsioon osaleb. Tuleb vaadata nii neid suhteid kus organisatsioon on kliendi (sõltuja) rollis kui ka neid kus organisatsioon on teenusepakkuja rollis. Näiteks raamatukogu puhul on organisatsioon teenusepakkuja rollis lugejatele. Seega üheks organisatsiooni väliseks pädevusalaks on *lugeja.*

Rahvaraamatukogu on rahvaraamatukogude seaduse kohaselt kohaliku omavalitsuse asutus (https://www.riigiteataja.ee/akt/12798044?leiaKehtiv). Raamatukogu peab andma oma tegevusest kohalikule omavalitsusele aru. Selles suhtes on raamatukogu kliendi rollis ja üheks väliseks pädevusalaks on *kohalik omavalitsus*.

Reaalsuses võib üks ametikoht koondada endas mitmeid erinevaid rolle ning üks inimene töötada korraga mitmel erineval töökohal (st omada kõiki ametikoha poolt määratud rolle).



Näiteks tulenevalt sellest, et õppejõud töötab dotsendi ametikohal on tal ÕISi mõttes rollid *"õppejõud – IDU"* ja *"õppeakavad ja -ained"*. Tulenevalt sellest, et õppejõud oli lisaks õppetooli juhataja kohusetäitja, oli tal lisaks roll *"õppetool(küsitlused) – IDU"*. ÕISi mõttes on need kolm erinevat pädevusala. Selles mõttes on ametikohtade kasutamine pädevusalade leidmiseks esialgne, kuid sageli mitte piisav lahendus.

1.5Viga 5 (liiga pinnapealne pädevusalade valik)

Pädevusalad on "töötaja pädevusala" ja "juhataja pädevusala".

Kommentaar: Juhataja on samuti töötaja. Lisaks vaata eelmist probleemi.

1.6Viga 6 (puudub omanik)

Puudub omaniku pädevusala.

Kommentaar: Ettevõtete puhul on üheks pädevusalaks omanik. Omanik ja juhataja võivad, kui ei pruugi, olla üks ja sama isik. See ei oma ka tähtsust, sest pädevusalade analüüs vaatleb *rolle*. Üks osapool võib omada ühte või mitut rolli. Riigiettevõtte puhul on omanikuks riik (https://riigiteataja.ee/akt/24040).

1.7Viga 7 (funktsionaalne allsüsteem=roll)

Funktsionaalsete allsüsteemidena on esitatud erinevate rollide nimekiri. Näiteks: "administraatori, treeneri, juhataja ja kliendi funktsionaalsed allsüsteemid".

Kommentaar: Funktsionaalsed allsüsteemid on valesti leitud. Funktsionaalne allsüsteem peab vastama ühele süsteemi põhifunktsioonile. Antud juhul aga on süsteemi tükeldanud organisatsioonis kasutusel olevate rollide järgi.

Lihtsamal juhul vastab registri infosüsteemis (infosüsteem, mille põhieesmärk on andmete haldamine) igale registrile eraldi funktsionaalne allsüsteem. Funktsionaalne allsüsteem võimaldab läbi viia sellesse registrisse andmete lisamist, andmete vaatamist (aruanded), andmete muutmist ja andmete arhiveerimist/kustutamist. Ühte funktsionaalset allsüsteemi kasutavad ühe või mitme pädevusala esindajad. Register omakorda vastab ühele süsteemi põhiolemitüübile.

Näiteks kaupluse infosüsteemi mõned põhiolemitüübid on kaup, müük, arve ja töötaja. Vastavalt on siis kaupluse infosüsteemis:

- kaupade register,
- müükide register,
- arvete register,
- töötajate register.

ning:

- kaupade funktsionaalne allsüsteem,
- müükide funktsionaalne allsüsteem,
- arvete funktsionaalne allsüsteem,
- töötajate funktsionaalne allsüsteem (see võimaldab hallata töötajate isikuandmeid ja töötamise kohta käivaid andmeid).

Pädevusala esindaja töökoha funktsionaalsus on tema poolt kasutatavate funktsionaalse allsüsteemi teenuste summa.

1.8Viga 8 (viitamise puudumine)

Kui kasutate projekti tegemiseks fragmente kellegi teise töödest (nt mustreid), siis tuleb neile tekstis viidata. Vastasel juhul on tegemist plagiaadiga ja sellist tööd ei arvestata.

1.9Viga 9 (allsüsteemide halvad nimed)

Allsüsteemidele on valitud liiga spetsiifilised nimed.

Kommentaar: Näiteks lillepoes "lillede funktsionaalne allsüsteem", toidupoes "toidu funktsionaalne allsüsteem", raamatupoes "raamatute funktsionaalne allsüsteem". Saab teha üldistuse. Tegemist on "müügiartiklite funktsionaalse allsüsteemiga" või "kaupade funktsionaalse allsüsteemiga" või "toodete funktsionaalse allsüsteemiga". See allsüsteem teenindab müügiartiklite registrit või kaupade registrit või toodete registrit. Samas on väga oluline, et olemitüüpide/atribuutide semantika kirjelduses kasutataks lisaks üldmõistetele (nt müügiartikkel, kaup, toode) ka valdkonnaspetsiifilisi mõisteid (nt lilled, toidukaubad).

Kindlasti kasutage kogu projektis ühesugust terminoloogiat. Eelneva näite korral otsustage näiteks, et kasutate terminit "kaup“, millest tulenevalt on infosüsteemis kaupade funktsionaalne allsüsteem ja kaupade register. Rääkige kogu projektis kaupadest (mitte toodetest või müügiartiklitest). Olemitüüpide/atribuutide semantika kirjelduses, mis on Teie projekti sõnastikuks, saate kirjutada, et "kaup" korral on alternatiivsed terminid "toode" ja "müügiartikkel".

1.10Viga 10 (registrite ebasobiv skoop)

Iga olemitüübi kohta on leitud erinev register.

Kommentaar: Register vastab ühele *põhi*olemitüübile ja sellega seotud olemitüüpidele, mis moodustavad ühe loogilise terviku. Seega näiteks pole mõistlik teha eraldi registreid nagu:

- Tellimuse seisundi liikide register,
- Kauba seisundi liikide register,
- Kauba liikide register,
- Kliendi liikide register.

Selle asemel võiks kirjeldada *Klassifikaatorite registri*, millesse kuuluvad olemitüüpidele *Tellimuse seisundi liik*, *Kauba seisundi liik*, *Kauba liik* ja *Kliendi liik* vastavad andmed.

1.11Viga 11 (puudulik süsteemianalüüs)

Süsteemi pole sisuliselt läbi mõeldud. Näiteks laosüsteemis puudub andmebaasis võimalus registreerida laoseisu, selle muutust, toodete hindu, kauba partiisid, hindade muutuseid, toodete kategooriaid, soodustusi, makseviise, rahalaekumisi jne. Kurbnaljakas näide – üliõpilased projekteerisid panga infosüsteemi, mis võimaldab küll raha kontole kandmist, aga mitte raha kliendile väljastamist (pank kui *must auk*). Või teine näide – toidupoe või raamatupoe (ei ole e-pood) infosüsteemis nähakse ette kliendi isikuandmete registreerimine enne igat ostu. Vaevalt et inimesed sellisesse poodi ostu sooritama tuleksid. Või siis nähakse ette,

et klientide kohustuslikuks unikaalseks identifikaatoriks on Eesti isikukood – sellega piirate võimalike klientide hulka Eesti isikukoodi omanikega.

Kommentaar: (vaata ka soovitust nr 1) Reaalse elu infosüsteemid ongi keerulised. Pole midagi imekspandavat, kui mõne reaalse elu infosüsteemi ärireeglite kirjeldus on 4–5 lehekülge tihedas kirjas teksti. Tasub uurida mustreid (vt teema nr 8), tutvuda rohkem Internetis väljas olevate vastavate firmade kodulehekülgedega ja valida oma õppekavasse kindlasti majanduse aineid. Süsteemianalüüsi töö ei sobi inimesele, kes ei suuda süveneda detailidesse ja neid piinliku täpsusega uurida ja kirjeldada.

Valitud allsüsteemi tuleks projekteerida selliselt, et see võimaldaks koguda kõiki raamatupidamist antud valdkonnas huvitavaid andmeid.

1.12Viga 12 (puuduv juhtimistasand)

Süsteemi projekteerimisel pole võetud arvesse juhtimistasandit.

Kommentaar: Märgatav on, et paljud süsteemid tegelevad küll andmete registreerimisega, kuid need andmed ei leia kasutust keskmise ja pikema perioodiga juhtimisotsustuste vastuvõtmisel. Juhtkonnale pole nähtud ette võimalust isegi lihtsamate aruannete vaatamiseks.

Sageli tahetakse kogutud andmed kohe, kui neid enam operatiivtaseme juhtimises vaja ei lähe, andmebaasist ära kustutada. Sellisel juhul tuleks nad eelnevalt laadida andmeaida andmebaasi, mille põhjal saab teha pikemat ajaperioodi hõlmavaid päringuid ning andmeid analüüsida. Näiteks kindlasti pakub kaupluse omanikule huvi, kui palju on muutunud käive ja kasum ajaperioodi (nt viie või kümne aasta jooksul). Kaupadega tegelejale pakub näiteks kindlasti huvi, kui palju on muutunud ühe või teise kauba /kaubagrupi nõudlus või milliste kaubamärkidega tooteid ostjad eelistavad. Selliste andmete põhjal saab andmekaevandamise meetodeid kasutades teha tulevikuprognoose. Kuid selliste prognooside tegemiseks peavad andmetest olema piisavalt pikad aegread.

Samas võib teatud juhtudel andmete kustutamist nõuda seadus või eetilised tõekspidamised. Näiteks isikuandmete kaitse seadus sätestab, et "§ 17. Isikuandmete säilitamine (4) Kui isikuandmete säilitamise tähtaeg lõpeb, on vastutav ja volitatud töötleja kohustatud isikuandmed jäädavalt kustutama." Samuti sätestab seadus, et isikul on õigus nõuda enda isikuandmete kustutamist.

Oluline on ka kliendisuhete haldus (*Customer Relationship Management – CRM*). Näiteks kui andmebaasis on talletatud andmed kõigi püsikliendi ostude kohta, siis nende alusel saab teha kindlaks, milliseid tooteid/kaubamärke klient eelistab. Eelistuste alusel saab teha kliendile personaalseid pakkumisi. Samas peab kliendilt küsima selleks nõusolekut ja vaikimisi eeldama, et klient seda nõusolekut ei anna. See on näide isikuandmete kaitse seaduse poolt nõutavast vaikimisi andmekaitse põhimõtte rakendamisest, mille kohaselt pakutakse isikule vaikimisi kõige privaatsussõbralikumat ja kõige väiksemate ohtudega lahendust. "Suuremaid ohte

sisaldavad valikud peaksid olema need, mida inimene ise valib vaikelahenduse asemele." (Isikuandmete töötleja üldjuhend, 2018)

1.13Viga 13 (valesti valitud aruanded)

Juhtkonnale esitatavates aruannetes on detailandmed, mitte koondandmed.

Kommentaar: Detailandmetele lisaks (või nende asemel) vajavad juhid aruandeid, mis sisaldavad koondandmeid (statistikat) ning erinevate perioodide tulemuste võrdlust. See võimaldab neil võtta vastu tulevikku puudutavaid otsuseid. Juhtkonna esindajal ei pruugi olla vaja vaadata iga üksiku tellimuse detailseid andmeid. Samas pakub talle kindlasti huvi, kui suures kogusummas tehti tellimusi viimase kuue kuu jooksul ning kui palju muutus see summa võrreldes eelnevate perioodidega.

1.14Viga 14 (seaduste mitte tundmine)

Süsteemi loomisel ei arvestata kehtivate seadustega.

Kommentaar: Seadused on üheks oluliseks infosüsteemile esitatavate nõuete allikaks. Mõned infosüsteemide (sealhulgas andmebaaside) valdkonda reguleerivad olulised seadused.

- Avaliku teabe seadus.
- Digitaalallkirja seadus.
- Isikuandmete kaitse seadus. Seoses Euroopa andmekaitse reformiga hakati Euroopas sh Eestis alates 25. maist 2018 kohaldama Euroopa Parlamendis heaks isikuandmete kaitse üldmäärust. Eesti isikuandmete kaitse seadus sai sellega seoses uue sisu. Selle seaduse uus redaktsioon hakkas kehtima 15.01.2019. Uue seadusega võetakse üle isikuandmete kaitse üldmäärus.

Seaduste mitte tundmine ei vabasta nende täitmisest! Seaduste mitte tundmine ei vabasta karistusest!

"**Isikuandmed** on teave inimese ehk füüsilise isiku (andmesubjekti) kohta, millega teda saab otse või kaude tuvastada: nimi, isikukood, asukohateave, võrguidentifikaatorid (tunnused, mis sidevõrgus aitavad viia konkreetse isikuni), samuti füüsilised, geneetilised, vaimsed, majanduslikud, kultuurilised ja mistahes muud tuvastamist võimaldavad tunnused ja nende kombinatsioonid.

Eriliiki isikuandmed on andmed, millest ilmneb rassiline või etniline päritolu, poliitilised vaated, usulised või filosoofilised veendumused või ametiühingusse kuulumine, geneetilised andmed, isiku kordumatuks tuvastamiseks kasutatavad biomeetrilised andmed, terviseandmed ja andmeid seksuaalelu ja seksuaalse sättumuse kohta.

Isikuandmete töötlemine on andmetega tehtav mistahes toiming: kogumine, korrastamine, säilitamine, muutmine, lugemine, kasutamine, edastamine, ühendamine, kustutamine jne" (Isikuandmete töötleja üldjuhend, 2018)

"§ 14. Isikuandmete töötlemise põhimõtted

1) seaduslikkus ja õiglus – isikuandmeid töödeldakse seaduslikult ja õiglaselt;
2) eesmärgikohasus – isikuandmeid kogutakse täpselt ja selgelt kindlaksmääratud õiguspärastel eesmärkidel ning neid ei töödelda viisil, mis on nende eesmärkidega vastuolus;
3) kvaliteet – isikuandmed peavad olema piisavad ja asjakohased ning ei tohi olla ülemäärased andmetöötluse eesmärke arvestades;
4) õigsus – isikuandmed peavad olema õiged ja vajaduse korral ajakohastatud; mõistlike meetmetega tagatakse, et andmetöötluse eesmärgi seisukohast ebaõiged isikuandmed kustutatakse või parandatakse viivitamata;
5) säilitamine – isikuandmeid säilitatakse kujul, mis võimaldab andmesubjekti tuvastada üksnes seni, kuni see on vajalik selle eesmärgi täitmiseks, milleks isikuandmeid töödeldakse;
6) turvalisus – isikuandmeid töödeldakse viisil, mis tagab nende turvalisuse, sealhulgas kaitseb loata või ebaseadusliku töötlemise eest ning juhusliku kadumise, hävimise või kahjustumise eest, rakendades asjakohaseid tehnilisi või korralduslikke meetmeid." (Isikuandmete kaitse seadus)

Näiteks kui kavandate kaupluse andmebaasi projekti, siis tuleb selle seaduse valguses läbi mõelda, milliseid andmeid kliendi kohta kogutakse ja milliste andmete kogumine on tema privaatsuse liigne rikkumine. Eriliiki isikuandmete kogumine on lubatud ainult siis, kui see on rangelt vajalik ning üksnes isikuandmete kaitse seaduses sätestatud juhtudel.

1.15Viga 15 (mudeli halb kvaliteet)

Mudel on halva kvaliteediga.

Kommentaar: Järgnev jutt kehtib kõigi mudelite kohta. See kehtib nii tekstikirjelduste, kui ka diagrammide kohta. Mudelit ei tule koostada selleks, et endale (kliendi, ülemuse, õppejõu silmis) **"linnuke"** kirja saada, vaid selleks, et aidata endal ja teistel käsitletavast probleemist ja pakutavast lahendusest paremini aru saada. Linnukese jaoks tehtud mudelid on raiskamine. Nagu ütlevad paljud moodsad arendusmeetodid ja ka terve talupojamõistus, siis raiskamisest tuleb lahti saada.

Mudel on kommunikatsioonivahend. Kui see on mingil põhjusel lugejale arusaamatu, siis tema väärtus läheneb nullile. Miks võib mudel olla arusaamatu?

1. See ei kanna endas sisulist infot. See ei ütle kasutajale midagi uut. Selle asemel, et probleemi ja lahendust selgitada, tekitab see uut segadust ja arusaamatust. Analoogiks on rakendus, mis ei tee midagi paremini, kui turul juba olemasolevad rakendused.

2. Mudel võib küll olla sisuliselt korralik, aga keegi ei saa sellest aru, sest see on esitatud äärmiselt halvasti loetavalt. Näiteks A4 lehele paigutatud olemi-suhte diagrammile on kuhjatud mitukümmend olemitüüpi. Nende vahel on tihe seosetüüpide võrgustik, kusjuures palju on kattuvaid ja ristuvaid jooni. Samamoodi on halvasti loetav "ühes jorus" kirjutatud pikk tekst, kus pole kasutatud teksti struktureerimise võimalusi (taandread, tärnimine, olulise teksti esiletõstmine jne). Analoogiks on rakendus, mis võib küll kasutajate soovitavaid funktsioone täita, kuid mille kasutajaliidese kasutatavus on halb.

Näide: Kontseptuaalsesse andmemudelisse kirja pandud kitsenduste põhjal leitakse kitsendused, mida hakatakse andmebaasis realiseerima. Võrrelge, kummast atribuudi definitsioonist on lihtsam kitsendused üles leida.

Halvem:

Olemitüübi nimi	Atribuudi nimi	Atribuudi definitsioon
Isik	e_mail	Unikaalne. Aadress, millele saab üle võrgu (ühest arvutist või tööjaamast teise) saata isikule mõeldud kirjalikke sõnumeid kusjuures aadressis peab sisalduma @ märk. Väärtus ei tohi puududa.

Parem:

Olemitüübi nimi	Atribuudi nimi	Atribuudi definitsioon
Isik	e_mail	Aadress, millele saab üle võrgu (ühest arvutist või tööjaamast teise) saata isikule mõeldud kirjalikke sõnumeid. {Isiku unikaalne identifikaator. Registreerimine on kohustuslik. e_mail peab sisaldama "@" märki.}

Mida selgemalt on kitsendused dokumendis välja toodud, seda tõenäolisem on, et andmebaasi ehitajal ei jää need kahe silma vahele.

Nimetatud vigu aitab avastada enda koostatud dokumentide *inspekteerimine.* Inspekteerimise eesmärgiks on juhtida tähelepanu vigadele. Viga on standardi rikkumine või mittekooskõla teiste dokumentidega. Järgnevalt kirjeldatakse võimalikku inspekteerimise protsessi:

Projekti autor vahetab oma projekti mingi teise projekti autoriga. Autor (inspektor) hindab teise autori projekti. Hindamine toimub individuaalselt. Hindamiseks tuleb kasutada küsimustikku, mis esitatakse allpool.

Inspekteerija koostab järgnevad tulemid.

- Nimekiri kõigist leitud vigadest, tuues välja vaid põhilised märksõnad.

- Iga vea puhul võib määrata raskusastme (kerge, keskmine, raske).
- Küsimused, mida soovitakse autorilt küsida.

Seejärel toimub *vigade esitamise kohtumine* (*defect logging meeting*). Selle käigus loetletakse projekti autorile üles leitud vead.

- Ei arutata, kuidas viga parandada. See on projekt autori ülesanne. Kiirkorras tehtud parandused ei pruugi olla kõige kvaliteetsemad.
- Ei arutata, kas viga tegelikult eksisteerib või mitte.
- Autor ei kaitse oma projekti. Ta võib vaid vastata küsimustele.

SAADUD INFORMATSIOON ON ALUSEKS PROJEKTI PARANDAMISELE AUTORI POOLT. Kasu annab ka oma projekti ise inspekteerimine.

Lisamaterjalide lehe dokumendis (kasutajanimi: SIS2 ja parool: SIS2)
https://maurus.ttu.ee/download.php?aine=346&document=31568&tyyp=do
on kirjeldatud:

- mõnede enam kasutatavate mudelitüüpide inspekteerimiseks kasutatavad küsimustikud,
- mustrid (juhendid) selle kohta, kuidas koostada võimalikult hästi loetavaid diagramme.

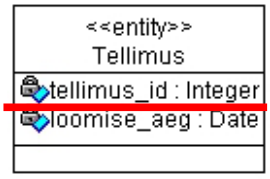
Lisamärkus: Süsteemi allsüsteemideks jagamise üheks kõrvaltulemuseks on see, et ühe hiiglasliku diagrammi asemel luuakse mitu väiksemat ja loetavamat diagrammi (iga allsüsteemi kohta üks või mitu). Näiteks olemi-suhte diagrammide puhul näidatakse igal diagrammil vaid konkreetse registri olemitüübid, seosetüübid ja atribuudid + teiste registrite otseselt seotud olemitüübid ilma liigsete detailideta. Selleks, et saada ülevaade kogu andmebaasist, võib üldvaates esitada koond-diagrammi, millel on esitatud ainult põhiobjektidele vastavad olemitüübid ja seosetüübid.

1.16Viga 16 (surrogaatvõtmed kontseptuaalses andmemudelis)

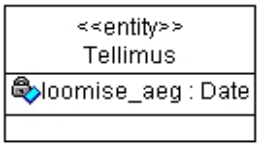
Kontseptuaalses andmemudelis on esitatud surrogaatvõtme atribuudid.

Kommentaar: Analüüsi täpsusega andmemudelis ei tuleks näidata surrogaatvõtmeid (atribuute a'la *tootaja_id*, *kaup_id, tellimus_id*), mille väärtusteks on süsteemi poolt genereeritavad unikaalsed täisarvud, mis on mõeldud andmete identifitseerimiseks *ainult* andmebaasis. Kõik atribuudid, mis selles mudelis esitatakse, peaksid kandma kasutaja jaoks sisulist tähendust – nende väärtuseid kasutatakse ka väljaspool tarkvara, st nende väärtuseid kuvatakse kasutajaliidese kaudu kasutajale. Kasutajad teavad neid väärtuseid ja võivad nende alusel olemitele ka väljaspool tarkvara viidata (rääkisin täna kolleeg 24323-ga ja ta ütles, et kaupa 2121 tuleb juurde tellida).

Vale (eeldusel, et *tellimus_id* võimalikud väärtused on süsteemi-genereeritud väärtused, mida väljapool tarkvara ei kasutata).



Õige:



Teisalt, kui jõuate prototüüpi luues arusaamisele, et kasutajatel on vaja *tellimus_id* väärtust näha ja nad hakkavad selle alusel tellimusi identifitseerima, siis tuleb kontseptuaalses andmemudelis kirjeldada atribuut *tellimus_id* või *tellimuse_kood* või *tellimuse_nr*.

1.17Viga 17 (välisvõtmed kontseptuaalses andmemudelis)

Kontseptuaalses andmemudelis on olemitüüpidele leitud välisvõtme atribuudid, mida detailanalüüsi käigus ei tule leida.

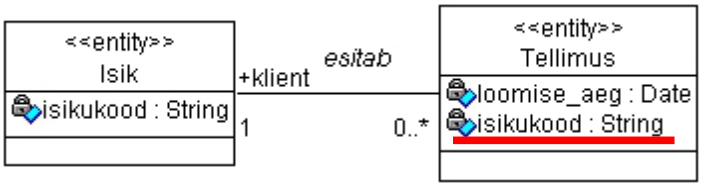
Kommentaar: Olemi-suhte diagrammid on ajalooliselt välja kujunenud kui abivahend SQL-andmebaaside projekteerimiseks. Olemi-suhte diagrammide koostamiseks on aegade jooksul välja töötatud mitmeid erinevaid meetodeid. Osades neist näidatakse välisvõtmeid, osades mitte.

Siin kursuses käsitletakse olemi-suhte diagramme kui osa kontseptuaalsest, semantilisest andmemudelist, mis kirjeldab andmebaasi vabana igasugustest tehnilistest kaalutlustest. ***Seega ei esitata kontseptuaalses andmemudelis ka välisvõtme atribuute***. Seosetüübid modelleeritakse olemi-suhte diagrammil olemtüüpide vaheliste seostena.

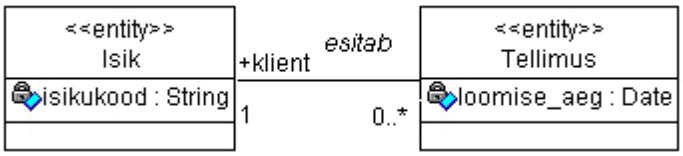
Välisvõtmete kaudu luuakse seos tabelites sisalduvate andmete vahel SQL aluseks oleva andmemudeli alusel loodud andmebaasis. Kuid peale SQL aluseks oleva andmemudeli on ka teisi andmebaasi andmemudeleid (nt hierarhiline, dokumendipõhine, võrkmudel, graafipõhine, võtme-väärtuse paarid, objektorienteeritud, objekt-relatsiooniline). Analüütiku ülesanne ei ole võtta vastu disainiotsuseid. Kasutatav andmebaasi andmemudel valitakse lõplikult välja disaineri poolt loogilise disaini käigus. Järelikult välisvõtme atribuute kontseptuaalses andmemudelis ei näidata. Diagrammil kahe olemitüübi vahel esineva seosetüübi näitamiseks piisab sellest, kui joonistatakse nende olemitüübi sümbolite vahele seost (*association*) tähistav joon ja määratakse seosetüübi omadused (võimsustikud, rollid, nimi).

See, et kontseptuaalne andmemudel on vaba tehnilistest kaalutlustest, tähendab, et sellest on võimalik erinevate teisenduste abil saada erinevatel andmemudelitel põhinevate andmebaaside tehnilised kirjeldused. Näiteks ühe teisenduste komplekti alusel saan kontseptuaalsest andmemudelist objektorienteeritud andmebaasi struktuuri kirjelduse ja teise teisenduste komplekti alusel saan samast kontseptuaalsest andmemudelist SQL-andmebaasi tabelite kirjelduse.

Vale.



Õige.



1.18Viga 18 (puuduvad üldistuste hulgad)

Olemi-suhte diagrammis ei ole esitatud üldistusseoste hulkade kohta käivaid kitsendusi.

Kommentaar: Üldistusseoste hulga (*generalization set*) juures tuleb kirjeldada kitsendused, mis seda täpsustavad. Meenutan, et ka ühte elementi sisaldav hulk on hulk.

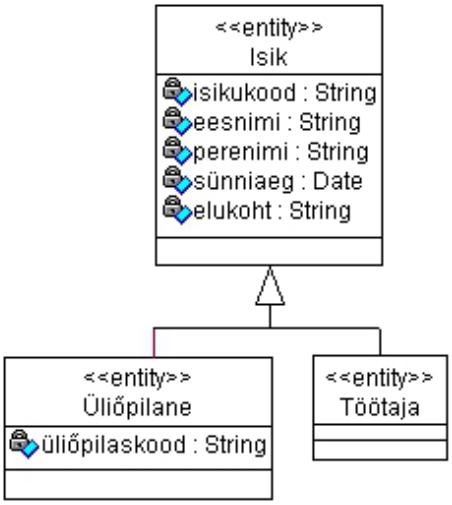
Osaluskohustus määrab, kas iga ülatüüpi kuuluv olem peab kuuluma ka mõnda alamtüüpi.

- Kui iga ülatüüpi kuuluv olem peab kuuluma vähemalt ühte alamtüüpi, siis tähistatakse seda diagrammil **{Mandatory}**.
- Kui iga ülatüüpi kuuluv olem ei pea kuuluma mõnda alamtüüpi (see tähendab, et võib leiduda ülatüüpi kuuluvaid olemeid, mis ei kuulu ühtegi alamtüüpi), siis tähistatakse seda diagrammil **{Optional}**.

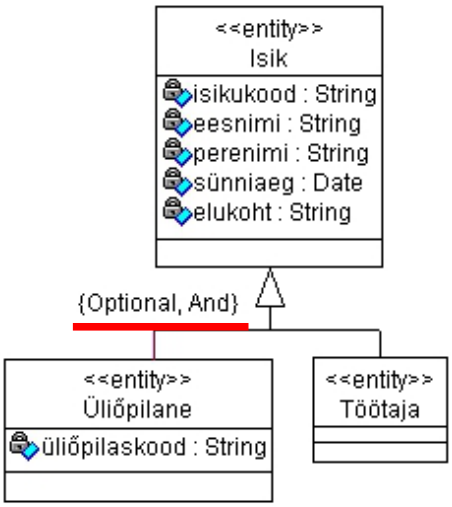
Kuuluvus määrab maksimaalse alamtüüpide arvu, kuhu ülatüüpi olem võib kuuluda.

- Kui iga ülatüüpi olem saab kuuluda maksimaalselt ühte alamtüüpi, siis tähistatakse seda diagrammil **{Or}**.
- Kui iga ülatüüpi olem saab kuuluda rohkem kui ühte erinevasse alamtüüpi, siis tähistatakse seda diagrammil **{And}**.

Halvem.



Parem.



Viimasel diagrammil esitatud kitsendused ütlevad järgmist.

- Võib leiduda isikuid, kes ei ole ei üliõpilased, ega ka töötajad **{Optional}**.
- Võib leiduda isikuid, kes on nii üliõpilased, kui ka töötajad **{And}**.

Taolised kitsendused tulenevad reeglitest, mis kehtivad organisatsioonis, millele infosüsteemi luuakse.

Märkus: Rational Rose CASE vahendis tuleb sellised kitsendused kirjutada diagrammile vabatekstina.

1.19Viga 19 (kasutusjuhtude vale jaotus)

Funktsionaalset allsüsteemi kirjeldavate kasutusjuhtude all on kirjas ka selliseid, mis kuuluvad mingisse teise allsüsteemi. Näiteks valitakse töös käsitlemiseks müükide funktsionaalne allsüsteem. Kasutusjuhtudesse aga pannakse kirja ka klientide registreerimise, töötajate registreerimise ja töötajate palkade üle arve pidamise kasutusjuhtumid.

Kommentaar: Iga kasutusjuhu juures tuleb kriitiliselt mõelda, kas see on antud (funktsionaalse) allsüsteemi kasutusjuht või kuulub mõnda teise allsüsteemi, mis samuti on allsüsteemide nimekirjas nimetatud. Kasutusjuhu samasuguse täpsusega kirjelduse dokumendis mitmes kohas esitamine pole mõistlik (kui sellist kirjeldust on vaja muuta, siis tuleb teha muudatus paljudes kohtades; kui kõiki koopiaid ei muudeta, siis pole kirjeldus enam kooskõlaline; lisaks sellele kasvatab dubleerimine dokumendi mahtu). Seega tuleks dokumendis alati kasutusjuht üks kord kirja panna ja vajadusel sellele viidata. Analoogiana mõelge sellele, et täpselt samasugused probleemid on andmebaasis andmete liiasusega.

Kui kasutusjuht on allsüsteemide ülene (näiteks *kasutaja tuvastamine*), siis tuleb see märkusena kirja panna ning dokumendis seda samuti mitte dubleerida. Sellise kasutusjuhu võib CASE vahendis panna eraldi paketti "Läbivad huvid".

1.20Viga 20 (kasutusjuhtude vale skoop)

Kasutusjuht ei vasta elementaarse äriprotsessile.

Kommentaar: Tuleb leida ja kirjeldada kasutusjuhud, millest igaüks vastab mingile elementaarsele äriprotsessile. Iga selline protsess viiakse läbi ühe tegutseja poolt ühel ajahetkel ja ühes kohas. Iga selline kasutusjuht käivitub reaktsioonina mingile sündmusele, selle läbimine annab tegutsejale väärtuse ning selle läbimise järel on süsteemis terviklikud andmed. Lisaks tuleb kirja panna just süsteemi kasutamise juhtumid, st protsessi läbiviimiseks kasutatakse süsteemi abi.

Sellest soovitusest tulenevalt ei kirjelda järgnevad punktid sobivaid kasutusjuhte.

- Lepingu üle läbirääkimine (see võtab tunde/päevi/nädalaid/kuid/aastaid, see võib toimuda erinevatel aegadel ja erinevates kohtades ning selles võivad osaleda erinevad osapooled)
- Võileiva tegemine (see ei hõlma arvutisüsteemi kasutamist)
- Vali e-poes tellimuse tegemise käigus makse tüüp (kliendi eesmärgiks on esitada tellimus ning saada huvipakkuvad kaubad. Makse tüübi valimine on sellest väike osa).

Järgnevad on sobilikud kasutusjuhud.

- Registreeri leping
- Loe retsepte
- Esita tellimus

Selleks, et kontrollida kasutusjuhu sobivat teralisust, võib kasutada *kohvitassi testi*. Iga kasutusjuht on selline, et selle läbimise järel võiks tegutseja lubada endale tassi kohvi (loodame, et kohvitamise kõrvalt jääb ka aega tööd teha).

1.21Viga 21 (kasutusjuhtude halvad nimed)

Kasutusjuhu nimi ei viita tegevusele.

Kommentaar: Kasutusjuht kirjeldab elementaarset äriprotsessi. Kasutusjuhu nimi peab viitama tegevusele. Viimane variant kõlab aktiivsemalt ja tegutsejaid rohkem kaasa haaravamalt.

Halb: Kaupade koondaruanne

Parem: Kaupade koondaruande vaatamine

Veel parem: Vaata kaupade koondaruannet

1.22Viga 22 (puuduvad viited andmebaasioperatsioonidele)

Kasutusjuhtude laiendatud formaadis tekstikirjeldustest ei viidata andmebaasioperatsioonidele.

Kommentaar: Kasutusjuhtude laiendatud formaadis tekstikirjeldused läheksid liiga mahukaks, kui seal hakataks detailselt kirjeldama kasutaja tegevuse tulemusel käivitunud süsteemi andmebaasi kasutust. Lõppkasutajale, kes kasutusjuhte üle vaatab või isegi kirjutada aitab, on selline kirjeldus juba liiga tehniline. Süsteemi kasutusjuht peaks kirjeldama süsteemi funktsionaalsust kasutaja vaatepunktist, mitte süsteemi sisemise toimimise põhimõtteid.

Andmebaasi kasutuse kirjeldused pannakse kirja andmebaasioperatsioonide lepingutesse ning kasutusjuhtude tekstikirjelduses lisatakse süsteemi tegevuste juurde vaid viited nende operatsioonide lepingutele. Kui andmebaasis toimuvad muudatused on kasutusjuhu täitmise stsenaariumist ja järeltingimustest arusaadavad, siis ei pea kõigi operatsioonide kohta lepinguid looma. Lepingud luuakse vaid kõige keerukamate/olulisemate operatsioonide kohta. Antud kursuses tuleb lepingud luua kõigi operatsioonide kohta, mille käigus toimub andmete modifitseerimine (lisamine, muutmine või kustutamine).

Põhimõtteliselt võib lepingud luua ka lugemisoperatsioonide kohta. Siis oleks järeltingimustes kirjeldatud, millised andmed operatsiooni tulemusena leitakse. See oleks iseenesest kasulik, sest erinevad kasutusjuhud võivad oma toimimiseks vajada samasuguste andmete lugemist ja selle kirjeldusse igasse kasutusjuhtu lisamine oleks dubleerimine. Antud kursuse projektis pole siiski vaja selliste operatsioonide lepinguid lahti kirjutada.

1.23Viga 23 (ebatäpsed andmebaasioperatsioonide järeltingimused)

Andmebaasioperatsioonide lepingu järeltingimused ei näita andmebaasis toimunud muudatusi või ei tee seda piisavalt täpselt. Näide: "Järeltingimus: Tellimuse andmed salvestati andmebaasi".

Kommentaar: Üks või mitu andmebaasioperatsiooni moodustavad teenuse, mida register pakub oma kasutajatele. Iga operatsioon eraldi on elementaarteenus, kuid erinevaid elementaarteenuseid saab kombineerida keerukamateks teenusteks.

Teenust võib kasutada infosüsteem, mille koosseisu register kuulub. Teenust võib pakkuda ka välistele kasutajatele (teine infosüsteem), mis täidavad teenuse kasutamiseks vajalikud tingimused (nt piisav turvatase).

Kui andmebaasioperatsioon teeb muudatuse andmebaasis, siis andmebaasioperatsioonide lepingute järeltingimustes tuleb täpselt kirjeldada olemid/atribuudid/seosed, millele vastavad andmed operatsiooni käigus loodi/muudeti/kustutati. Larman (2002) märgib, et sageli unustatakse just olemite vaheliste seoste kohta käivate andmete loomise/muutmise/kustutamise kirjeldamine.

Näide: Kauba registreerimise operatsioon.

```
Eeltingimused:
• Kauba_seisundi_liik eksemplar ksl (nimetus="Aktiivne")
on registreeritud
• Pakendi_tüüp eksemplar pt (pakendi tüübi
identifikaator) on registreeritud
Järeltingimused:
--Loo eksemplare
• Kaup eksemplar k on registreeritud
--Väärtusta atribuute
• k.kauba_kood:= p_kauba_kood
• k.nimetus:= p_nimetus
• k.hetke_hind:= p_hetke_hind
• k.kirjeldus:= p_kirjeldus
--Loo seoseid
• k ja ksl seos on registreeritud
• k ja pt seos on registreeritud
```

Larman (2002) kirjeldab järeltingimusi näitelava kujundit kasutades. Operatsiooni alguses langeb näitelavale (andmebaasile) eesriie. Eesriide taga toimub tegevus – näitlejad vahetavad asukohta, keegi tuleb lavale ning keegi lahkub lavalt, muudetakse dekoratsioone. Eesriie tõuseb ja näitelava (andmebaasi) seisund on muutunud. On loodud/muudetud/kustutatud olemite/atribuutide/seoste kohta käivaid andmeid. Järeltingimuses pannakse kirja näitelava (andmebaasi) seisundi muutused. Seda mis jäi samaks kirja ei panda.

Taolisel viisil järeltingimuste kirjeldamine annab ka võimaluse kontrollida, kas iga lepingut ikka on võimalik täita. Lepingut ei saa täita, kui kontseptuaalses andmemudelis pole kirjeldatud kõik operatsioonis viidatavad olemitüübid/ atribuudid/ seosetüübid.

1.24Viga 24 (andmebaasioperatsioonide ebasobiv skoop)

Andmebaasioperatsioonide lepingud kirjeldavad andmebaasiga mitteseotud küsimusi. Näiteks andmebaasioperatsiooni sisuks on kuvada tühi sisestusvorm.

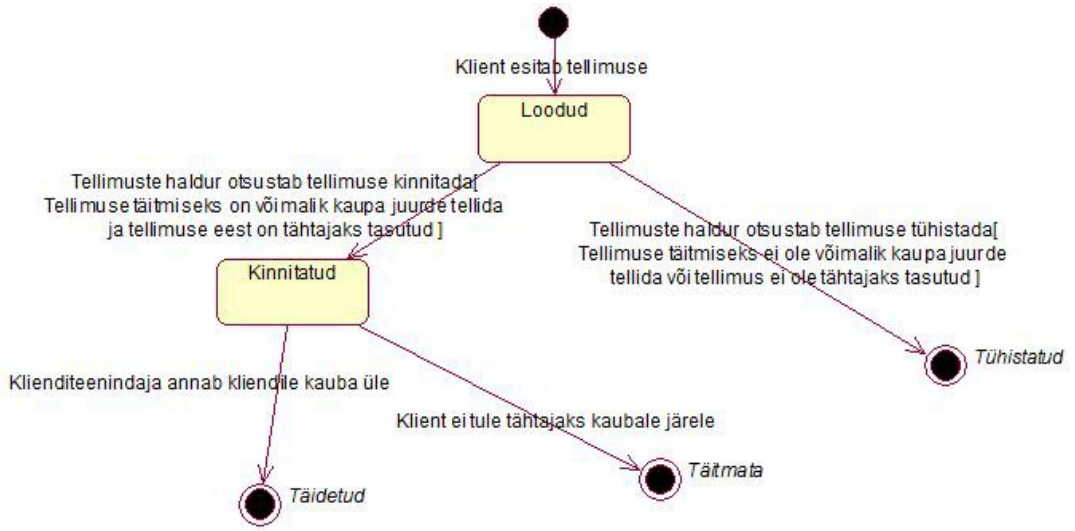
Kommentaar: Antud juhul on tegemist *süsteemi operatsiooni lepinguga*, mis on laiem mõiste kui andmebaasioperatsiooni leping. Andmebaasi- operatsioonide lepingud tuleb kirjutada operatsioonide kohta, mille käigus kuidagi andmebaasi kasutatakse (andmebaasioperatsioonid). Iga andmebaasioperatsioon on ka süsteemi operatsioon. Kuid iga süsteemi operatsioon ei pruugi olla andmebaasioperatsioon.

1.25Viga 25 (vead seisundidiagrammis)

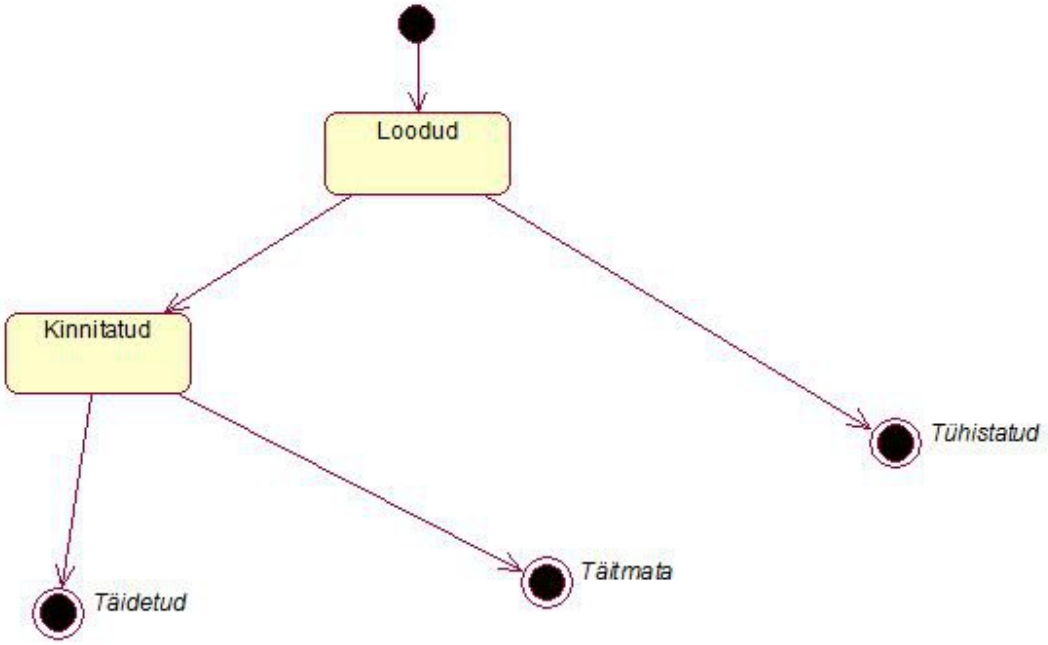
Seisundidiagrammis on vigu.

Kommentaar: Esitan järgnevalt erinevaid variante olemitüübi "Tellimus" seisundidiagrammist ja kommenteerin neid. Rõhutan, et tegemist on lihtsustatud diagrammiga ning reaalsete süsteemide korral võib see diagramm olla palju keerukam.

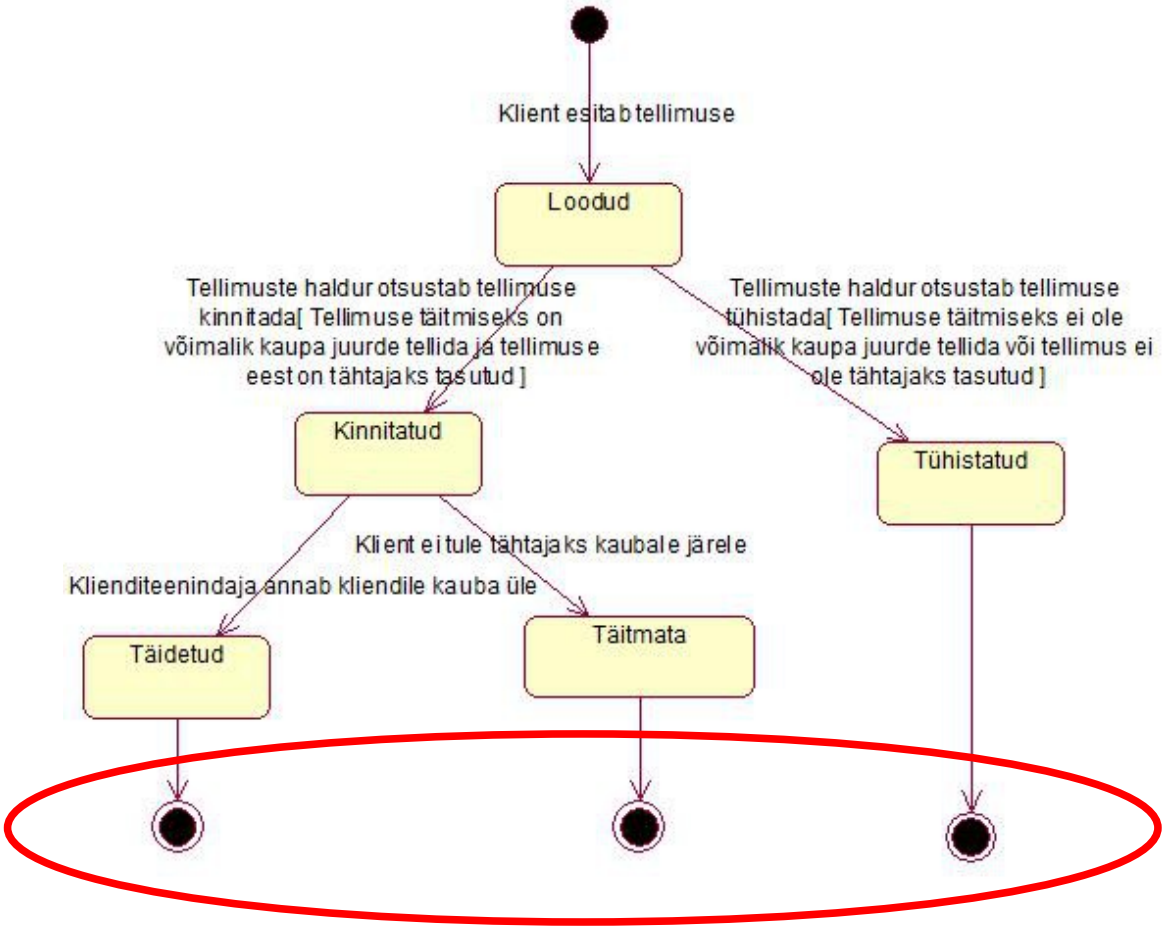
Parem.



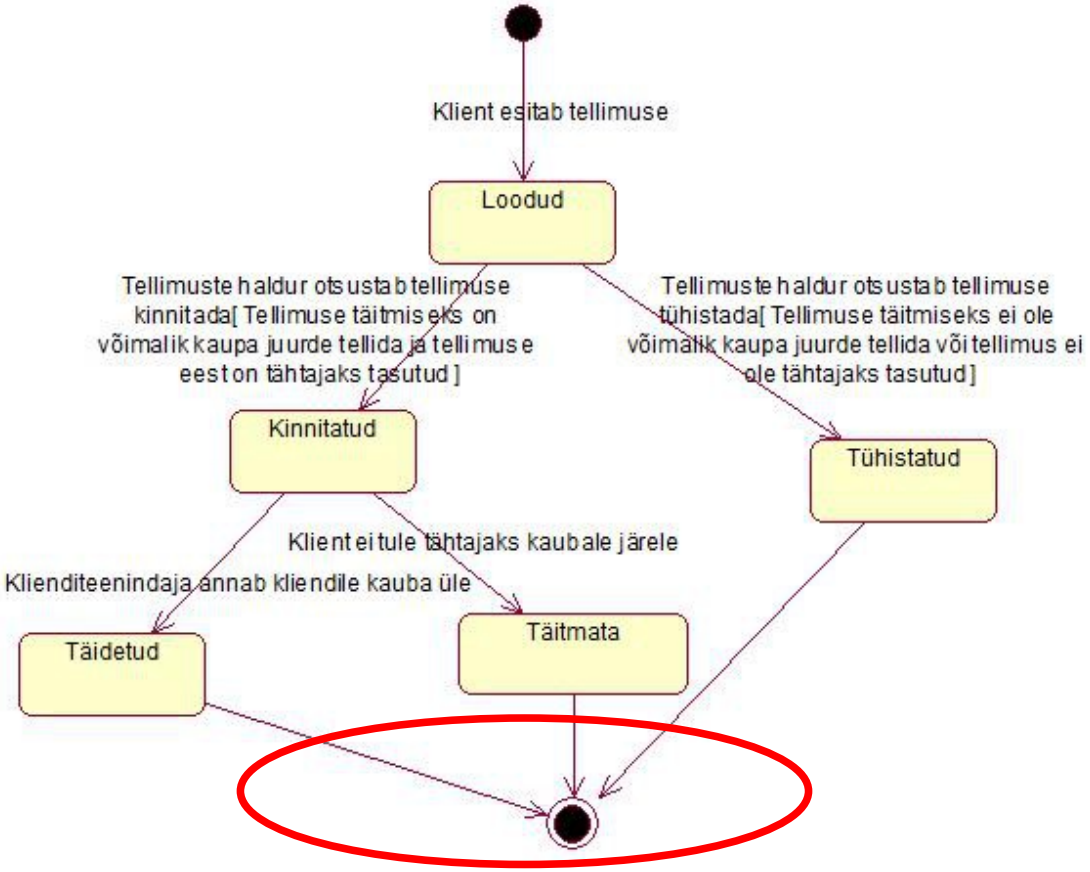
Halb. Mis viga? Seisundidiagrammil ei ole esitatud sündmuseid, mis põhjustavad seisundi muudatusi.


Loodud
Kinnitatud
Tühistatud
Täitmata
Täidetud

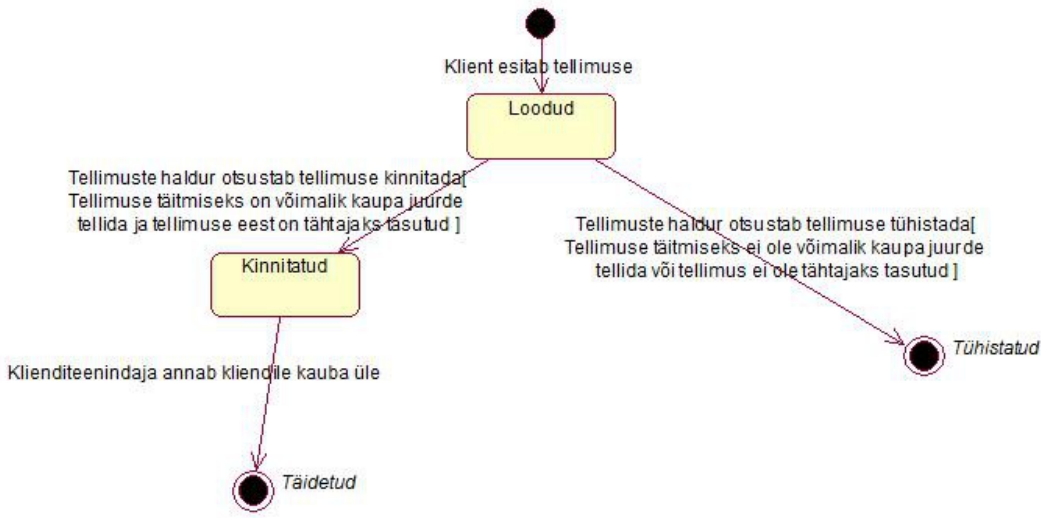
Halb. Mis viga? Lõppseisundid on valesti esitatud. Tegelikud lõppseisundid on "Täidetud", "Täitmata", "Tühistatud". Diagramm jätab aga näiteks mulje, et tellimus läheb seisundist "Täidetud" lõppseisundisse, kusjuures pole esitatud seisundi muudatust põhjustavat sündmust ega ka lõppseisundi nime.



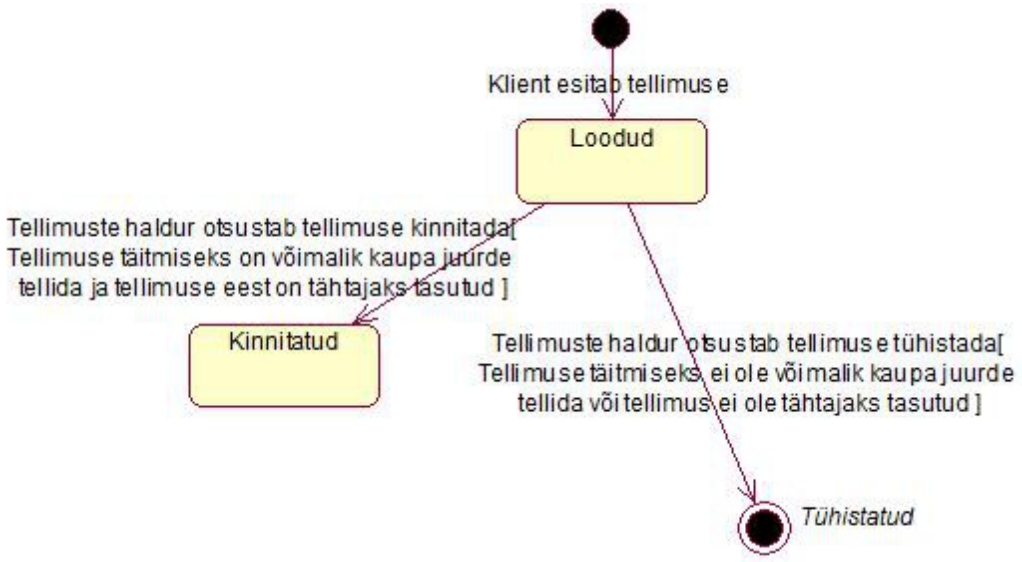
Halb. Mis viga? Lõppseisundid on valesti esitatud. Variatsioon eelmisest näitest.



Halb. Mis viga? Pole mõeldud läbi erijuhtumeid (näiteks, et klient ei tule tellitud kaubale järgi). Lihtne on mõelda tavastsenaariumile, kuid infosüsteemi edu/ebaedu määrab see, kui hästi suudab see toime tulla harvaesinevate olukordadega. Analüütik peab oskama selliseid olukordi leida.



Halb. Mis viga? Pole esitatud ühe olemitüübi kõikvõimalikke seisundeid. Olemitüübi seisundidiagrammis tuleb kirjeldada KÕIK selle olemitüübi seisundid, mitte alamosa nendest.



1.26Viga 26 (ei saa genereerida arveid)

Projekteeritakse tellimuste funktsionaalset allsüsteemi, aga ei nähta ette võimalust arvete genereerimiseks.

Kommentaar: Järgnevalt on esitatud arve näide.

Arve nr 345

Tellija: Ants Kask
Aadress: Pikk tn. 12-4, 13222 Tallinn

Koostamise aeg: 12.02.2026
Maksmise tähtaeg: 15.02.2026
Makseviis: ülekanne
a/a Rootsi pank: 22 893 435 2091
Viitenumber 9239354182

Nimetus	***Ühik***	***Kogus***	***Hind ****	***Summa ****
Visiitkaart	tükk	500	1 EUR	500 EUR
T-särk	tükk	10	5 EUR	50 EUR

Summa: * 550 EUR

* – käibemaksuga (24%)
****
AS Kõva Sõna
Aadress: Lai tn. 34, 12232 Tallinn
Äriregistri kood: 80667812
Telefon: +372 6734234
a/a: 22 893 435 2091

Arvest võib mõelda kui aruandest, mis on genereeritud tellimuse põhjal. Arve genereerimine on üks osa tellimuse koostamise kasutusjuhust. MS Accessis võib arve väljastamise realiseerida kasutades trükist (*report*).

Täpselt samuti tuleb näha ette tšeki genereerimine sularahaga müügitehingu juures.

2 Vead SQL-andmebaasis

Järgnevalt kirjeldatakse tüüpvigu SQL-andmebaasis. Näited põhinevad MS Accessi andmebaasisüsteemil, kuid vead on universaalsed ja võivad esineda mistahes SQL-andmebaasisüsteemi andmebaasides. Muide, MS Accessis saab luua ka CHECK kitsendusi. Valideerimisreeglite graafilises kasutajaliideses kirjeldamise eelis seisneb selles, et saab määrata veateateid, mida andmebaasis andmete muutjatele kitsenduste vastu eksides kuvatakse.

2.1Viga 27 (puuduvad unikaalsuse kitsendused)

Andmebaasis pole jõustatud kõiki vajalikke unikaalsuse kitsendusi.

Kommentaar: Tabelis võib olla mitu kandidaatvõtit. Hea disaini praktika kohaselt valitakse neist üks primaarvõtmeks ning ülejäänuid hakatakse nimetama alternatiivvõtmeteks. Nii primaarvõtmed, kui ka alternatiivvõtmed, tuleb andmebaasis jõustada.

Oletame, et MS Accessi andmebaasis tuleb luua tabel *Amet*.

Amet(ameti_kood, nimetus)
Primaarvõti (ameti_kood)
Alternatiivvõti (nimetus)

Tabeli disainivaates peab avanema järgnev pilt. Veeru *ameti_kood* ees on võtmekese sümbol, mis näitab, et see veerg kuulub primaarvõtmesse. Primaarvõtme kitsendus tähendab antud juhul, et tabelis *Amet* ei tohi olla kahte rida, milles on samasugune *ameti_kood* väärtus. Samuti peab tabeli *Amet* igas reas olema *ameti_kood* väärtus määratud (ei tohi olla NULL).

Field Name	Data Type
ameti_kood	Number
nimetus	Short Text

General | Lookup

Field Size	Byte
Format	
Decimal Places	Auto
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	
Required	Yes
Indexed	Yes (No Duplicates)
Text Align	General

Tabelis on ka üks alternatiivvõti – (nimetus). Selle jõustamiseks olen deklareerinud veerule unikaalsuse kitsenduse (seda näitab *Indexed=Yes (No Duplicates)* – unikaalsuse kitsenduse jõustamiseks loodi unikaalne indeks). Samuti on deklareeritud, et veerg nimetus on kohustuslik (*Required=Yes*).

Field Name	Data Type
ameti_kood	Number
nimetus	Short Text

General | Lookup

Field Size	50
Format	
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	
Required	Yes
Allow Zero Length	No
Indexed	Yes (No Duplicates)
Unicode Compression	No
IME Mode	No Control
IME Sentence Mode	None

Võtmed (unikaalsuse nõuded) tulevad valdkonna nõuetest. Võib juhtuda, et nimetus (üksinda) ei ole unikaalne. Järgnev näide pärineb Tallinna Tehnikaülikooli ÕISist, kust võib leida sama nimega, kuid erineva ainekoodiga õppeaineid.

Õppeained

1-4 // 4

	ainekood	aine nimetus	aine nimetus ingl.k.
1.	VAY0481	Kehaline kasvatus I	Physical Education I
2.	VAY0482	Kehaline kasvatus I	Physical Education I
3.	VAY0490	Kehaline kasvatus II	Physical Education II
4.	VAY0500	Kehaline kasvatus II	Physical Education II

Samas, kui ma alternatiivvõtmetele tähelepanu ei pööra, siis võin tulemuseks saada sellise tabeli.

Tootaja

tootaja_id	ameti_kood	synni_kp	aadress	registr_aeg	on_aktuaalr	eesnimi	perenimi
1	1	3.01.1981	Tallinn	30.01.2006 14:00:00	True	Mart	Mets
2	1	3.01.1981	Tallinn	30.01.2006 14:00:00	True	Mart	Mets
3	1	3.01.1981	Tallinn	30.01.2006 14:00:00	True	Mart	Mets
4	1	3.01.1981	Tallinn	30.01.2006 14:00:00	True	Mart	Mets
5	1	3.01.1981	Tallinn	30.01.2006 14:00:00	True	Mart	Mets
6	1	3.01.1981	Tallinn	30.01.2006 14:00:00	True	Mart	Mets
7	1	3.01.1981	Tallinn	30.01.2006 14:00:00	True	Mart	Mets

Vormiliselt on kõik korras ja igas reas on unikaalne *tootaja_id* väärtus. Kuid sisuliselt olen sama töötaja andmed mitmekordselt registreerinud. See suurendab andmemahte ja aeglustab päringuid ning muudab raskemaks päringute tulemuste interpreteerimise.

Kui tabelis on jõustatud ainult üks võti ja see on surrogaatvõti (süsteemi genereeritud unikaalne täisarvuline väärtus), siis võib uute olemite registreerimisel kergesti tekkida olukord, et andmed lisatakse tabelisse mitmekordselt. Kuidas see on võimalik? Kasutaja sisestab vormi uue olemi andmed ja annab korralduse salvestada. Rakendus saadab korralduse ära, kuid enne kui saab andmebaasisüsteemilt vastuse ühendus katkeb. Veebirakenduse korral võib lisaks juhtuda, et veebilehitseja saadab korralduse ära, kuid enne kui saab veebiserverilt vastuse ühendus katkeb (nt ebastabiilse mobiiliühenduse tõttu). Samuti võib kasutajaliides olla lihtsalt halvasti disainitud, nii, et see ei anna salvestamise õnnestumisest või ebaõnnestumisest kuidagi märku.

On mis on, kasutajale jääb mulje, et tema salvestamise soov jäi täitmata ja ta vajutab uuesti salvestamise nupule. Selle tulemusena lisatakse olemi andmed tabelisse teistkordselt, ainult võtmeväärtus genereeritakse rea jaoks uus. Kui tabelis on üks või mitu sisulist tähendusega võtit, siis uuesti salvestamisel tekib primaarvõtme või unikaalsuse kitsenduse viga ja olemi andmeid ei lisata tabelisse mitmekordselt.

2.2Viga 28 (puuduvad välisvõtme kitsendused)

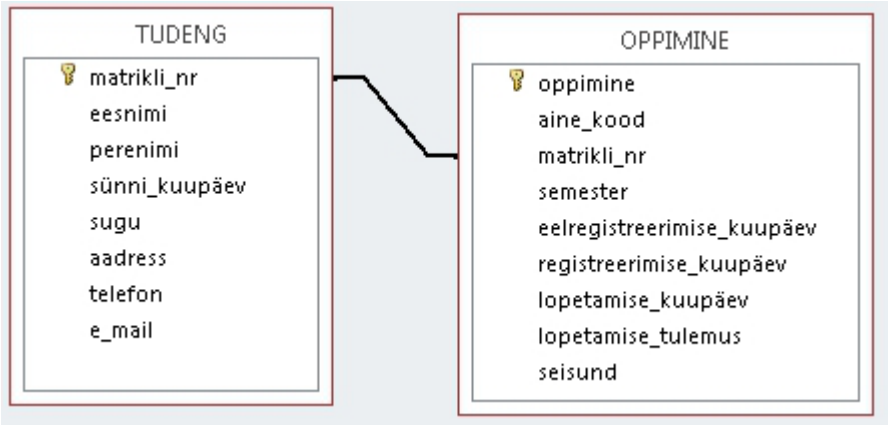
Andmebaasis ei ole jõustatud välisvõtme kitsendusi ning seega ei kontrolli andmebaasisüsteem viidete terviklikkuse reegli täidetust.

Kommentaar: MS Accessis tuleb välisvõtme kitsenduse graafilise defineerimise aknas teha linnukese valiku *Enforce Referential Integrity* ette. Sellega jõustatakse viidete terviklikkuse kontroll. Viidete terviklikkus (*referential integrity*) tähendab, et välisvõtme veerus tohivad ainult olla seotud tabeli võtmeveerus olevad väärtused. Kui see kontroll on jõustatud, siis andmebaasisüsteem ei lase enam välisvõtme veergu ebasobivaid väärtuseid kirjutada. Välisvõtmete puhul tuleb olla ettevaatlik vaikimisi väärtustega. Vaikimisi väärtus ei tohi minna vastuollu viidete terviklikkuse reegliga.

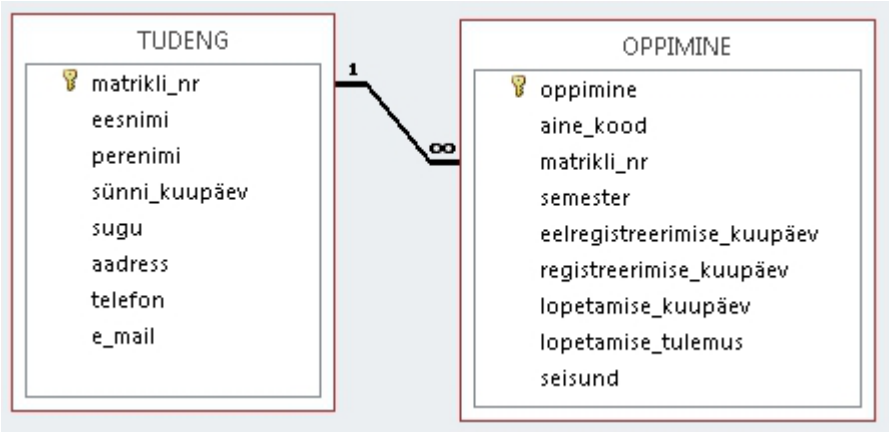
Näide: Oletame, et tabelis *Oppimine* on välisvõtme veerg *matrikli_nr*, mille abil luuakse seos tabelis *Tudeng* olevate andmetega. Järgnevalt vaatame, milline pilt võib avaneda MS Accessi *relationship view* (seoste vaate) alt.

Välisvõtme kitsendus pole andmebaasis defineeritud (**halb!**).

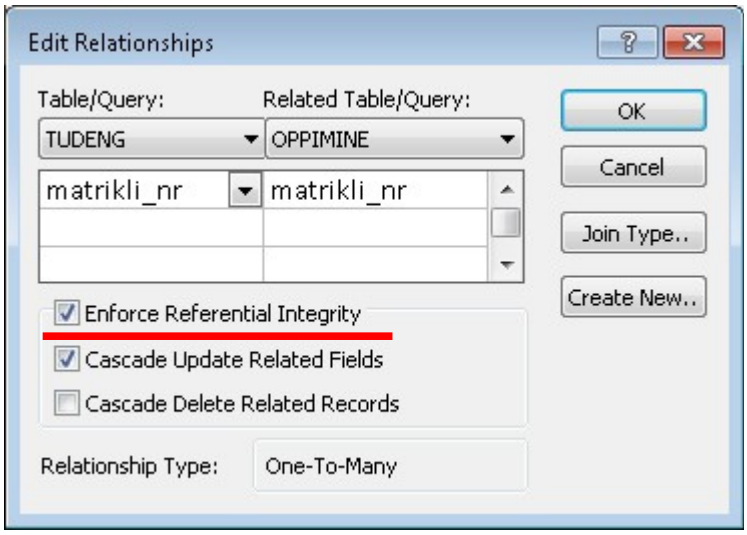
Välisvõtme kitsendus pole andmebaasis defineeritud. Andmebaasis on kirjeldatud kahe tabeli vaheline seos, kuid viidete terviklikkuse reegli kontrollimine pole jõustatud (**halb!**).



Välisvõtme kitsendus on korrektselt defineeritud ja andmebaasis on jõustatud sellega seoses viidete terviklikkuse reegli kontrollimine (õige ja soovitav tulemus) (**hea!**).



Kui teen topeltklõpsu välisvõtit tähistaval joonel, siis avaneb aken.



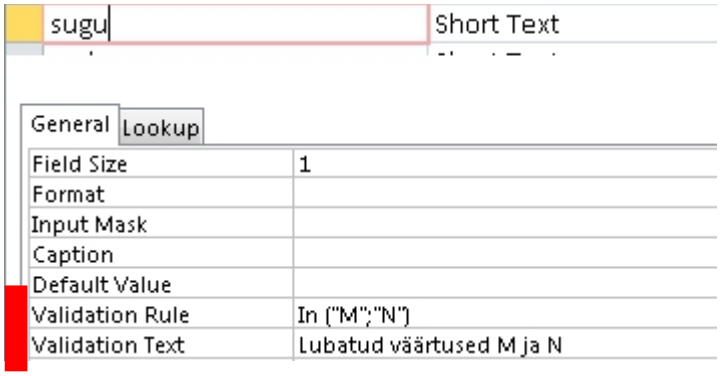
Cascade Update Related Fields ees olev linnuke tähendab antud juhul, et matrikli numbri muutmisel tabelis *Tudeng* muudetakse seda automaatselt ka tabelis *Oppimine* (kui tudengil leidub mõni seotud õppimine).

2.3Viga 29 (puuduvad CHECK kitsendused)

Andmebaasis pole jõustatud kõiki vajalikke CHECK kitsendusi.

Kommentaar: Väga oluline on, et andmebaasis oleksid jõustatud kitsendused, mis ei lase sinna panna ebaloogilisi andmeid. Ilmselt ebaloogiliste andmete näited: kauba hind või tellimuse kogus on negatiivsed ning tellimus tehti Madisepäeva lahinguga sama ajal. MS Accessis saab CHECK kitsenduste asemel kasutada veeru või tabeliga seotud valideerimisreegleid (*validation rule*). Sellise kitsenduse võib siduda üksiku veeruga (kui soovitakse kontrollida ainult selles veerus olevaid andmeid) või terve tabeliga (kui soovitakse realiseerida kontrolle, mis hõlmavad mitut tabeli veergu). **Kõik need kitsendused tulenevad kontseptuaalsesse andmemudelisse kirja pandud kitsendustest (st ärge unustage neid kitsendusi olemitüüpide ja atribuutide definitsiooni kirja panna).**

Järgnevalt on esitatud kaks näidet valideerimisreeglite kohta MS Accessis. Esimene valideerimisreegel on seotud veeruga *sugu* ja määrab, et selles veerus on lubatud väärtused M ja N. *Validation Text* väärtus on veateade, mida kuvatakse reeglitele mittevastavate andmete registreerimisel.



Teine reegel on tabelitaseme reegel, mis on seotud tabeliga *Opetamine* ja määrab, et lõpukuupäev ei tohi olla väiksem alguskuupäevast. *lopukuupaev* ja *alguskuupaev* on veerud tabelis *Opetamine*.

NB! Nii nagu teisedki SQL-andmebaasisüsteemid võimaldab MS Access deklareerida CHECK kitsendusi CREATE TABLE või ALTER TABLE lausete abil. Valideerimisreegli eeliseks on võimalus ära määrata veateade (*validation text*). Andmebaasi hallatavuse

seisukohast oleks parem, kui kõik sellised kitsendused on realiseeritud ühtemoodi – kas siis valideerimisreeglitega või CHECK kitsendustena.

2.4Viga 30 (klassifikaatori koode genereerib andmebaasisüsteem)

Klassifikaatorite koodid genereeritakse andmebaasisüsteemi poolt. Klassifikaatorite koodid on veerus, mis on ebasobivat tüüpi.

Kommentaar: Klassifikaatorite näited.
- Kliendi liik
- Kliendi seisundi liik (näide seisundiklassifikaatorist; need sõltuvad konkreetse ettevõtte äriprotsessidest)
- Amet
- Riik (https://en.wikipedia.org/wiki/List_of_ISO_3166_country_codes)
- Keel (https://en.wikipedia.org/wiki/List_of_ISO_639-1_codes)
- Valuuta (https://www.xe.com/currency/)
- Kauba liik

Mis on klassifikaator? Eesti Statistika kodulehel oli selle kohta kunagi hea selgitus: https://web.archive.org/web/20200728212634/https://www.stat.ee/klassifikaatorid

Klassifikaatorite nimistu Eesti statistikaameti kodulehel: https://klassifikaatorid.stat.ee/?siteLanguage=ee

Klassifikaatorite tabelis hoitavad andmed kirjeldavad mingis teises (tavaliselt palju suurema andmehulgaga) tabelis olevaid andmeid. Taoliste tabelite kasutamine aitab kontrollida kasutajate poolset andmete sisestust. Näiteks kasutaja saab registreerida tabelis *Klient* olevasse välisvõtme veergu ainult sellise kliendi liigi koodi, mis leidub tabelis *Kliendi_liik*.

Klassifikaatorite tabelites olevad andmed tuleb andmebaasis registreerida enne andmebaasi kasutuselevõttu lõppkasutajate poolt. Ilma nende andmeteta ei saa lõppkasutajad andmebaasi kasutada.

Juba kontseptuaalse andmebaasi projekteerimise käigus võib seda tüüpi andmetele vastavusse panna eraldi olemitüübi. Loogilise disaini käigus leitakse tabelid.

```
Kliendi_liik(kliendi_liigi_kood, nimetus, kirjeldus)
Primaarvõti (kliendi_liigi_kood)
Alternatiivvõti(nimetus);

Klient (klient_id, eesnimi, perenimi, kliendi_liig_kood)
Primaarvõti(klient_id)
Välisvõti (kliendi_liigi_kood) Viitab Kliendi_liik(kliendi_liigi_kood);
```

Klassifikaatorite tabelites hoitavad andmed muutuvad harva ja tuleks dokumenteerida juba andmebaasi projekteerimise käigus (nt loogilise disaini faasis).

Sellistes tabelites olevate andmete väljapakkumine on konkreetse allorganisatsiooni ülesanne, kuid nende kinnitamine ja sisestamine peaks olema andmete administraatori ja juhtkonna otsustada.

Seega ei peaks klassifikaatorite koodid (nt kliendi liigi kood) olema surrogaatvõtmed. Selliste koodide väärtused dokumenteeritakse ja sisestatakse andmebaasi volitatud kasutajate poolt (st koodi väärtust ei genereeri süsteem).

Miks on kasulik, et vastavate volitustega andmebaasi kasutajad saavad ise klassifikaatorite koode sisestada?

Näide: Kui on kindlalt teada, et kood 2 tähistab püsiklienti, siis saab püsiklientide andmete leidmiseks teha päringu:

```
SELECT * FROM Klient WHERE kliendi_liigi_kood=2;
```

selle asemel, et teha päring:

```
SELECT Klient.* FROM Klient INNER JOIN Kliendi_liik ON
Klient.kliendi_liigi_kood=Kliendi_liik.kliendi_liigi_kood
WHERE Kliendi_liik.nimetus='Püsiklient';
```

Näide: Kui on teada, et enamik registreeritavaid kliente on "tavalised" kliendid, keda tähistav kood on 1, siis saab deklareerida tabelis *Klient* veerule *kliendi_liigi_kood* vaikimisi väärtuse 1.

```
ALTER TABLE Klient ALTER COLUMN kliendi_liigi_kood SET
DEFAULT 1;
```

Samuti tuleb klassifikaatori koodidele vastavate veergude tüüpe. Näiteks tabelis *Kliendi_liik* on veerg *kliendi_liigi_kood*. Milline võiks olla selle veeru tüüp? Mõttesse tulevad täisarvutüübid INTEGER ja SMALLINT. Milline nendest tüüpidest valida?

Tüüpi INTEGER kuuluvad täisarvud vahemikus -2147483648 kuni +2147483647 (otspunktid kaasa arvatud).

Tüüpi SMALLINT kuuluvad täisarvud vahemikus -32768 kuni +32767 (otspunktid kaasa arvatud).

Andmebaasi disainer peaks mõtlema, milline on pikas perspektiivis erinevate kliendi koodide arv. Kui see arv jääb alla 32768, siis tuleks tüübiks valida SMALLINT.

MS Accessis võib klassifikaatorite koodide talletamise jaoks sageli kasutada veerge, mis on täisarvutüüpi *Byte.* Sellesse tüüpi kuuluvad väärtused 0–255 (otspunktid kaasa arvatud) ja iga väärtuse salvestamiseks kulub üks bait.

Field Name	Data Type
ameti_kood	Number
nimetus	Short Text

General | Lookup

Field Size	Byte

Sageli pole klassifikaatorite koodid üldse täisarvu tüüpi. Näiteks rahvusvaheline riikide klassifikaator (ISO 3166 Codes (Countries)) https://en.wikipedia.org/wiki/List_of_ISO_3166_country_codes

näeb ette, et iga riigi kohta kahetähelise koodi, kolmetähelise koodi ja kolmest numbrist koosneva koodi. Eesti puhul on need koodid vastavalt "EE", "EST", 233. Seega, kui hakkate looma tabelit *Riik*, siis pole mingit mõtet lisada sinna veergu a'la *riik_id*, kus on kasutaja väljamõeldud täisarvud. Selle asemel peab tabelis *Riik* primaarvõtme veeruks olema veerg, kus on rahvusvaheliselt väljatöötatud ja tunnustatud koodid.

2.5Viga 31 (halvasti nimetatud võtmeveerud)

Erinevate tabelite võtmeveergudel on samasugune nimi (näiteks "ID" või "kood").

Kommentaar:

Halb (disain "id").

```
Klient(id, eesnimi, perenimi)
Primaarvõti (id);

Tellimus (id, klient_id, summa, täitmise_tähtaeg)
Primaarvõti (id)
Välisvõti (klient_id) Viitab Klient (id);
```

Parem (disain "tabel_id").

```
Klient(klient_id, eesnimi, perenimi)
Primaarvõti (klient_id);

Tellimus (tellimus_id, klient_id, summa, täitmise_tähtaeg)
Primaarvõti (tellimus_id)
Välisvõti (klient_id) Viitab Klient (klient_id);
```

Miks on disain *"tabel_id"* parem?

Kokkuvõtlikult – lihtsamad päringud! Järgnevalt põhjendatakse seda koos näidetega.

Disaini *"id"* puhul võib näiteks klientide ja tellimuste andmete ühendamiseks teha päringu:

```
SELECT K.id, K.perenimi, T.id, T.summa
FROM Klient AS K INNER JOIN Tellimus AS T ON
K.id=T.klient_id;
```

Päringu tulemuseks olevas nimeta tabelis on kaks veergu, mille nimi on *id* (see on SQLi reeglitega kooskõlas).

```
 id | perenimi | id | summa
----+----------+----+--------
  1 | Tamm     |  1 | 100.00
(1 row)
```

Kuidas peaks päringu tulemuste vaataja aru saama, millises veerus on kliendi identifikaator ja millises tellimuse identifikaator? Ka päringu teksti lugejal tuleb pingutada, et aru saada, millised andmed on millises veerus. Samuti tekib probleem, kui rakenduses viidatakse päringu tulemuses olevale veerule kasutades veeru nime (*id*). Probleemi vältimiseks tuleks päringu tulemuses muuta veergude nimesid.

```
SELECT K.id AS klient_id, K.perenimi, T.id AS
tellimus_id, T.summa
FROM Klient AS K INNER JOIN Tellimus AS T ON
K.id=T.klient_id;
```

```
 klient_id | perenimi | tellimus_id | summa
-----------+----------+-------------+--------
         1 | Tamm     |           1 | 100.00
(1 row)
```

Disaini *"id"* korral tuleb ühendamise tingimus alati pikalt välja kirjutada (näiteks K.id=T.klient_id), sest veerud, üle mille ühendamine toimub, on erineva nimega:

```
SELECT K.id AS klient_id, K.perenimi, T.id AS
tellimus_id, T.summa
FROM Klient AS K INNER JOIN Tellimus AS T ON
K.id=T.klient_id;
```

Samas disaini *"tabel_id"* korral saab kasutada lühemat kirjapilti:

```
SELECT klient_id, perenimi, tellimus_id, summa
FROM Klient INNER JOIN Tellimus USING (klient_id);
```

või isegi:

```
SELECT klient_id, perenimi, tellimus_id, summa
FROM Klient NATURAL JOIN Tellimus;
```

Viimasel juhul toimub ühendamine üle samanimeliste veergude.

Kui tahan andmeid kõikidest ühendatavate tabelite veergudest, siis disaini *"tabel_id"* korral saan kirjutada:

```
SELECT *
FROM Klient INNER JOIN Tellimus USING (klient_id);
```

või:

```
SELECT *
FROM Klient NATURAL JOIN Tellimus;
```

Päringu tulemuses on veerg *klient_id* ühekordsel. Kuid kui kirjutan disaini *"id"* korral

```
SELECT *
FROM Klient AS K INNER JOIN Tellimus AS T ON
K.id=T.klient_id;
```

, siis on tulemuses kaks veergu, kus on kliendi identifikaator.

Lisaks lühidusele on USING kirjapildi eeliseks see, et vähendab võimalust, et tabelid ühendatakse üle valede veergude (st tulemus on vale).

Vale:

```
SELECT K.klient_id, K.perenimi, T.tellimus_id, T.summa
FROM Klient AS K INNER JOIN Tellimus AS T ON
K.id=T.kaup_id;
```

NB! USING ja NATURAL JOIN paraku MS Accessis kasutada ei saa.

2.6Viga 32 (tühjade stringide lubamine tekstiväljas)

Kontseptuaalses andmemudelis on tekstilise atribuudi definitsioonis kirja pandud kitsendus, et atribuudi väärtus ei tohi olla tühi string ("")). Andmebaas on realiseeritud MS Accessis ja sellele atribuudile vastavas veerud on tühjad stringid lubatud (Allow Zero Length = Yes).

Kommentaar:

Näide:
Ametikoha nimetus ei tohi olla tühi string.

Vale.

Field Name	Data Type
ameti_kood	Number
nimetus	Short Text

General | Lookup

Field Size	50
Format	
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	
Required	Yes
Allow Zero Length	Yes

Õige.

Field Name	Data Type
ameti_kood	Number
nimetus	Short Text

General | Lookup

Field Size	50
Format	
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	
Required	Yes
Allow Zero Length	No

NB! Veeru loomisel on vaikimisi määrang Allow Zero Length = Yes.

2.7Viga 33 (puuduvad veateated)

MS Accessi andmebaasisüsteemis loodud andmebaasis jõustatakse andmebaasi tasemel deklaratiivne kitsendus (luuakse veeru või tabeliga seotud *Validation Rule*), kuid ei kasutata MS Accessi pakutavat võimalust sõnastada veateade, mida kuvatakse, kui üritatakse registreerida kitsendusele mittevastavaid andmeid (jäetakse määramata *Validation Text*).

Kommentaar:

Halb.

Amet

Field Name	Data Type
ameti_kood	Number
nimetus	Short Text
max_tootunde_nadalas	Number
min_palk	Currency

General | Lookup

Field Size	Integer
Format	
Decimal Places	Auto
Input Mask	
Caption	
Default Value	40
Validation Rule	Between 0 And 50
Validation Text	

Amet

ameti_kood	nimetus	max_tootur	min_palk	Click to Add
1	administraator	100	700,00 €	
2	liini operaator	40	800,00 €	
		40		

Microsoft Access

One or more values are prohibited by the validation rule 'Between 0 And 50' set for 'Amet.max_tootunde_nadalas'. Enter a value that the expression for this field can accept.

OK Help

Parem.

Amet

Field Name	Data Type	Description
ameti_kood	Number	
nimetus	Short Text	
max_tootunde_nadalas	Number	
min_palk	Currency	

Field Properties

General | Lookup

Field Size	Integer
Format	
Decimal Places	Auto
Input Mask	
Caption	
Default Value	40
Validation Rule	Between 0 And 50
Validation Text	Maksimaalne töötundide arv nädalas peab olema vahemikus 0 – 50 (otspunktid kaasa arvatud)

Amet

ameti_kood	nimetus	max_tootur	min_palk	Click to Add
1	administraator	100	700,00 €	
2	liini operaator	40	800,00 €	
		40		

Microsoft Access

Maksimaalne töötundide arv nädalas peab olema vahemikus 0 – 50 (otspunktid kaasa arvatud)

OK Help

Süsteemide puhul, kus kasutatavas andmebaasisüsteemis ei saa andmebaasi kitsendusega niimoodi veateadet siduda, seisneb viga selles, et rakenduse poolt lõppkasutajale esitatav veateade on kasutajale arusaamatu või eksitav. Veateated on tarkvarasüsteemides üldiselt üks unarusse jäänud teema, mille kohta saab veidi lisainfot näiteks järgnevast bakalaureusetööst.

Põlluste, K.K., 2015. *SQL lausete veateadete headuse analüüs kolme erineva andmebaasisüsteemi näitel*. Bakalaureusetöö. TTÜ Informaatikainstituut [WWW] https://digi.lib.ttu.ee/i/?3455 (18.01.2026)

2.8Viga 34 (ebatäpsed veateated)

MS Accessi andmebaasisüsteemis loodud andmebaasis jõustatakse andmebaasi tasemel deklaratiivne kitsendus (luuakse veeru või tabeliga seotud *Validation Rule*). *Validation Text* abil realiseeritud veateade ei viita vea asukohale.

Kommentaar:

Halb.

Validation Text: 0– 50 (otspunktid kaasa arvatud)

Validation Text: Peab olema vahemikus 0– 50 (otspunktid kaasa arvatud)

Parem.

Validation Text: Maksimaalne töötundide arv nädalas peab olema vahemikus 0 – 50 (otspunktid kaasa arvatud)

2.9Viga 35 (ebareaalsed ajahetked – puuduvad CHECK kitsendused)

Lubate andmebaasis registreerida täiesti ebareaalseid sündmuste toimumise kuupäevi.

Kommentaar:

Näide: MS Accessis kuuluvad DATE tüüpi kuupäeva esitavad väärtused, kus on aasta vahemikus 100 kuni 9999.

Halb. Saate registreerida töötaja, kes sündis 2. veebruaril 1267 – pole probleemi.

Tootaja

Field Name	Data Type
tootaja_id	AutoNumber
ameti_kood	Number
eesnimi	Short Text
perenimi	Short Text
synni_aeg	Date/Time

General | Lookup

Format	Short Date
Input Mask	
Caption	
Default Value	
Validation Rule	
Validation Text	

tootaja_id	ameti_kood	eesnimi	perenimi	synni_aeg
1	1	Mart	Mets	2.02.1267

Parem. Kitsenduse abil on piiratud lubatud sünnikuupäevade hulka.

Tootaja

Field Name	Data Type
tootaja_id	AutoNumber
ameti_kood	Number
eesnimi	Short Text
perenimi	Short Text
synni_aeg	Date/Time

Field Properties

General | Lookup

Format	Short Date
Input Mask	
Caption	
Default Value	
Validation Rule	Between #1.01.1920# And #31.12.1999#
Validation Text	Sünni aeg peab olema vahemikus 1. jaanuar 1920 ja 31. detsember 1999 (otspunktid kaasa arvatud)

NB! Kõik sellised kitsendused tuleb juba kontseptuaalsesse andmemudelisse kirja panna.

2.10Viga 36 (CHECK kitsendused, mis ei arvesta aja kulgemisega)

Kuupäevadega seotud kitsenduste defineerimisel ei arvestata, et aeg voolab pidevalt ja katkematult.

Kommentaar: Kõik tabeli read peavad igal ajahetkel rahuldama kõiki sellele tabelile deklareeritud kitsendusi. Kui kitsendustes kasutatakse mittedeterministlikke funktsioone (nt hetke aja leidmise funktsioonid, mis tagastavad samade argumentidega erinevatel ajahetkedel erineva tulemuse), siis ei pruugi see nõue olla täidetud.

Näide: Infopäringute registreerimine.

Vale. Tabelisse rea lisamise hetkel rahuldab rida seda kitsendust. Kuid ka homme on see rida andmebaasis ja siis see rida enam seda kitsendust ei rahulda.

Infoparing

Field Name	Data Type
infop_id	AutoNumber
sisu	Short Text
loomise_aeg	Date/Time
klient_id	Number

General | Lookup

Format	
Input Mask	
Caption	
Default Value	
Validation Rule	>=Date()
Validation Text	Palun registeeri tänane kuupäev.

Õige. Sellisel veerul sellist kitsendust ei ole. Kui soovitakse kontrollida, et rea tabelisse lisamise hetkel peab kuupäev olema võrdne tänase kuupäevaga või tulevikus, siis tuleb seda kontrollida rakenduses või andmete lisamiseks mõeldud protseduuris.

2.11Viga 37 (puuduvad vaikimisi väärtused)

Puuduvad vajalikud vaikimisi väärtused.

Kommentaar:

Näide: Tabelid kauba ja selle hetkeseisundi registreerimiseks.

```
Kauba_seisundi_liik(kauba_seisundi_liik_kood, nimetus)
Primaarvõti (kauba_seisundi_liik_kood)
Alternatiivvõti (nimetus)

Kaup(kauba_kood, nimetus, hetke_hind, kauba_seisundi_liik_kood,
registreerimise_aeg)
Primaarvõti (kauba_kood)
Välisvõti (kauba_seisundi_liik_kood) Viitab
Kauba_seisundi_liik(kauba_seisundi_liik_kood)
```

Halb. Tabelis *Kaup* ei ole vaikimisi väärtuseid.

Parem. Tabeli *Kaup* veerul *kauba_seisundi_liik_kood* on vaikimisi väärtuseks selle seisundi kood, millesse kaup läheb (seisundidiagrammi alusel) kohe peale loomist. Tuleb hoolitseda selle eest, et tabelis *Kauba_seisundi_liik* on registreeritud sellise koodiga seisund.

Tabeli *Kaup* veerul registreerimise_aeg on vaikimisi väärtuseks pöördumine funktsiooni poole, mis tagastab hetke kuupäeva ja kellaaja.

2.12Viga 38 (ebakorrektsed vaikimisi väärtused)

Välisvõtmetel ebakorrektsed vaikimisi väärtused.

Kommentaar:

Näide: Tabelid sõiduautode registreerimiseks.

Vale. Välisvõtme veergudel on vaikimisi väärtused, millele puudub vastav primaarvõtme väärtus primaarses tabelis.

Sõiduauto

Käigukast_ID	Keretüüp_ID	Kütus_ID	Mark_ID	Vedav_sild_ID	Värv_ID
2	1	1	1	1	1
2	3	3	4	2	1
2	1	2	1	2	1
1	1	1	1	1	1
1	1	1	1	1	1
* 0	0	0	0	0	0

Õige. Nendel veergudel selliseid vaikimisi väärtuseid ei ole.

Vale. Primaarvõtme või unikaalsuse kitsendusega veerul on *staatiline* vaikimisi väärtus.

Vedav_sild

Vedav_sild_ID	nimetus
1	esisild
2	tagasild
3	AWD
* 0	

Õige. Nendel veergudel selliseid vaikimisi väärtuseid ei ole. Ei saa olla mitut rida, kus on samasugune võtme väärtus ja seega ei oma staatiline vaikimisi väärtus antud juhul mingit mõtet.

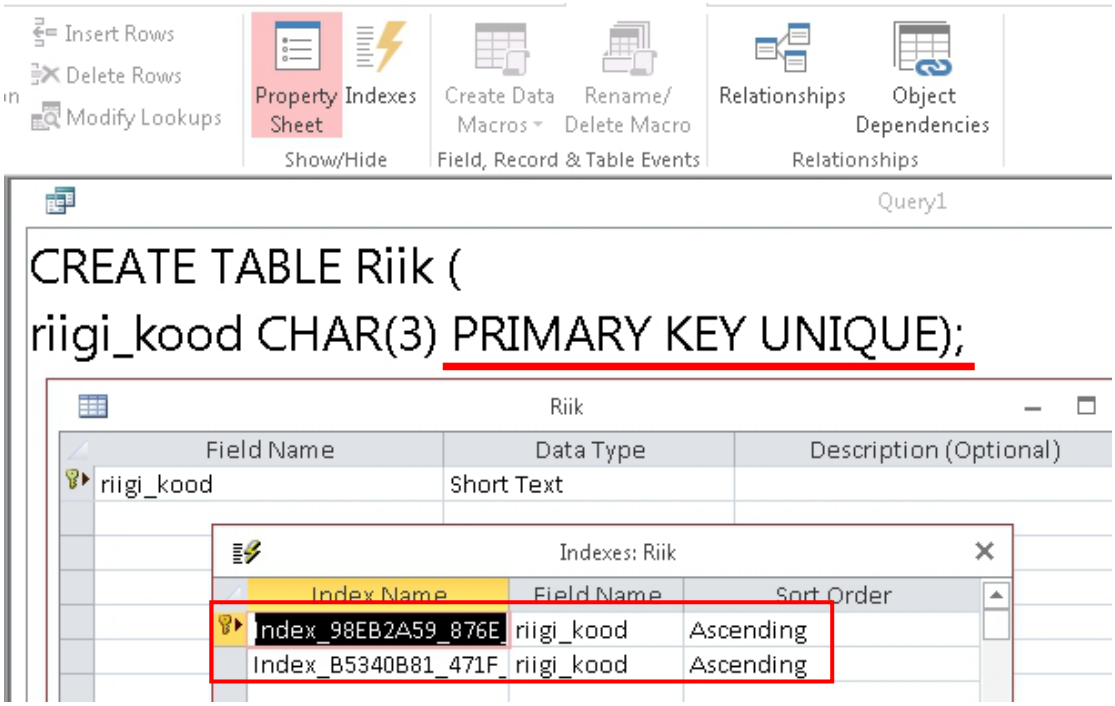
2.13Viga 39 (unikaalsuse kitsenduste asjatu dubleerimine)

Veerul on korraga nii primaarvõtme kui ka unikaalsuse kitsendus. Teiste sõnadega, primaarvõtmele lisaks määratakse samadele veergudele täiendav unikaalsuse kitsendus.

Kommentaar: Primaarvõti juba tähendab unikaalsust. Tulemuseks võivad olla üksteist dubleerivad indeksid (sõltub andmebaasisüsteemist). Mis kasu oleks Teile näiteks raamatust, mille lõpus on kaks ühesugust indeksit?

Näide: Klassifikaatori tabel *Riik*.

Vale.



Õige. Veerul *riigi_kood* on ainult PRIMARY KEY kitsendus.

2.14Viga 40 (üldine klassifikaatorite tabel)

Ühine klassifikaatorite tabel, kus on ka klassifikaatorite koodid ja nimetused.

Kommentaar: Küsimusele, miks see on probleem, vastatakse näiteks:

- tüüpiliste SQL-andmebaasi disainivigade nimekirja esimeses punktis (*Common Lookup Tables*): https://www.red-gate.com/simple-talk/sql/database-administration/five-simple--database-design-errors-you-should-avoid/
- SQL-andmebaaside eksperdi Joe Celko kirjutise alajaotises *One True look-up Table*: https://www.red-gate.com/simple-talk/databases/sql-server/t-sql-programming-sql-server/look-up-tables-in-sql/
- https://www.itprotoday.com/sql-server/designing-performance-lookup-tables
- https://www.apress.com/us/blog/all-blog-posts/best-practices-for-using-simple-lookup-tables/13323426
- https://www.sqlservercentral.com/articles/lookup-table-madness
  - Huvitav on lugeda ka selle artikli kommentaare: https://www.sqlservercentral.com/forums/topic/lookup-table-madness

Tasakaalu huvides on siin ka üks ühist klassifikaatorite tabelit toetav kirjutis:

- https://www.sqlservercentral.com/articles/a-lookup-strategy-defined

Sellel teemal kirjutab ka bakalaureusetöö:

Vellemaa, A.S., 2015. *Mõned disainimustrid klassifikaatorite esitamiseks SQL andmebaasides*. Bakalauruestöö. TTÜ Informaatikainstituut [WWW] https://digi.lib.ttu.ee/i/?3484 (18.01.2026)

Parem. Iga klassifikaatori tüübi kohta on loodud eraldi tabel ja seal on vähemasti klassifikaatorite koodid ja nimetused. Klassifikaatorite koodide kohta lugege **tüüpvea 30 kirjeldusest.**

2.15Viga 41 (kitsendustele annab nime andmebaasisüsteem)

Tabelite looja ei anna kitsendustele ise nimesid.

Kommentaar: Kas baastabelil on nimi? On. Kas veerul on nimi? On. Kas andmetüübil on nimi? On. Kas tabelis deklareeritud kitsendusel on nimi? On. Võin kirjutada tabeli loomise lause nii, et seda nime ise ei määra, kuid siis teeb seda andmebaasisüsteem. Erinevad süsteemid annavad nimesid erinevalt, kuid üldiselt on see nimi pikk, raske meeles pidada, halvasti kitsenduse tähendust edasi andev.

Näide:

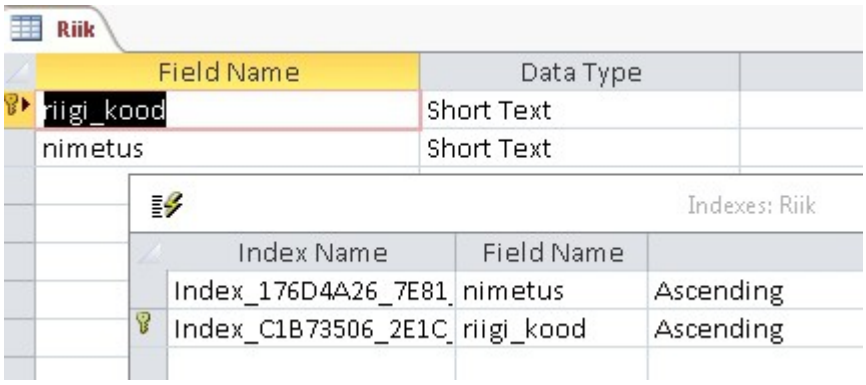
Halb.

```
CREATE TABLE Riik (
riigi_kood CHAR(3) NOT NULL PRIMARY KEY,
nimetus VARCHAR(50) NOT NULL UNIQUE);
```

Kitsenduste nimed:

- Index_176D4A26_7E81_46D5
- Index_C1B73506_2E1C_44CC

Näiteks, kui tahan kitsenduse SQL-lause abil eemaldada, siis pean teadma selle nime.

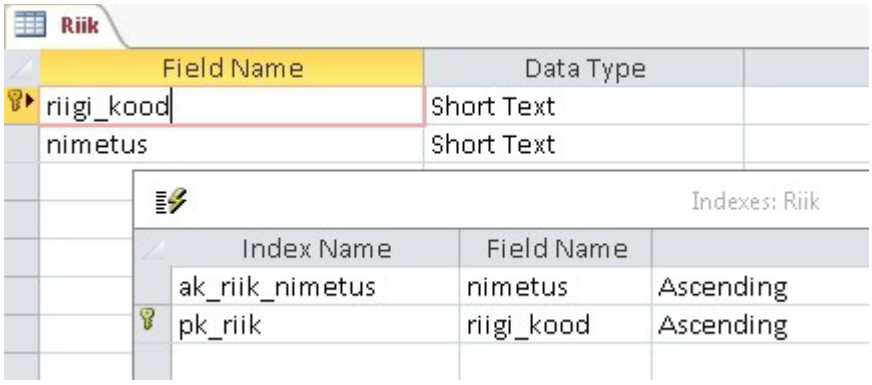


Parem.

```
CREATE TABLE Riik (
riigi_kood CHAR(3) NOT NULL,
nimetus VARCHAR(50) NOT NULL,
CONSTRAINT pk_riik PRIMARY KEY (riigi_kood),
CONSTRAINT ak_riik_nimetus UNIQUE (nimetus));
```

Kitsenduste nimed:

- pk_riik
- ak_riik_nimetus


Riik
Field Name
Data Type
riigi_kood
Short Text
nimetus
Short Text
Indexes: Riik
Index Name
Field Name
ak_riik_nimetus
nimetus
Ascending
pk_riik
riigi_kood
Ascending

2.16Viga 42 (ebasobivad kompenseerivad tegevused)

Välisvõtmetega seotud kompenseerivate tegevuste valik pole optimaalne.

Kommentaar: Viidete terviklikkuse kontrollimine peaks olema andmebaasisüsteemi ülesanne, et kellelgi poleks võimalust mingil viisil viidete terviklikkuse reeglist mööda hiilida. Andmebaasisüsteem saab reageerida viidete terviklikkuse vea tekkimisele nii, et muudab ise andmeid, selleks, et tulemuseks olevad andmed rahuldaksid viidete terviklikkuse reeglit.

Millal kasutada ON UPDATE CASCADE?

- Välisvõti viitab sisulise tähendusega võtmele. Sisulise tähendusega võtme väärtus omab andmebaasi kasutajatele tähendust, see sisestatakse inimkasutajate poolt ja seega võib olla vaja seda väärtust muuta. Sisulise tähendusega võtme veeru võib (kui veeru nimed on hästi antud) tunda ära selle järgi, et selle nimes on sõna "kood" (isikukood, riigi kood, äriregistri kood, üliõpilaskood jne).
  - Näiteks klassifikaatorite koodid on sisulise tähendusega võtmed ning nendele viitavad välisvõtmed peaksid kasutama ON UPDATE CASCADE.

Millal kasutada ON DELETE CASCADE?

- Välisvõti realiseerib kompositsiooni (nt *Tellimus-Tellimuse rida*).
- Välisvõti realiseerib üldistuse (nt *Isik-Töötaja).*
- Registri mitte-põhiobjekti alusel loodud tabelis olev välisvõti, mis viitab sama registri põhiobjekti alusel loodud tabelitele (nt *Kaup-Kauba_kategooria_omamine*).

Millal kasutada ON UPDATE SET NULL/ON DELETE SET NULL?

- Võib kaaluda, kui välisvõti lubab NULLe.
- Rusikareegel – mida vähem NULLe, seda parem; mida rohkem kohustuslikke veerge, seda parem. Seega SET NULL kompenseerivaid tegevusi peaks olema pigem vähem, kui rohkem.

Millal kasutada ON UPDATE SET DEFAULT/ON DELETE SET DEFAULT?

- Võib kaaluda, kui välisvõtme veerul on vaikimisi väärtus.
- Tuleb olla tähelepanelik, et see vaikimisi väärtus oleks viidatavas tabelis võtmeväärtus ning et sellist väärtust sealt tabelist ära ei kustutataks.
- Oletame, et süsteemist on vaja kustutada toiminguid teinud isikute andmed, kuid soovitakse, et info nende toimingute kohta süsteemis anonümiseeritult säiliks. Selleks võib süsteemis registreerida anonüümset isikut tähistava isiku (nt identifikaatoriga 0 või -1), määrata tema identifikaatori isiku tabeli veerule viitavale välisvõtme veerule vaikimisi väärtuseks ning kasutada välisvõtme deklaratsioonis ON DELETE SET DEFAULT.

2.17Viga 43 (CHECK kitsenduste dubleerimine)

Lisakitsenduste jõustamine välisvõtmete korral pole optimaalne.

Näide:

Oletame, et andmebaasis on tabelid:

Riik (riik_kood, nimetus)
Primaarvõti (riik_kood)
Alternatiivvõti (nimetus)

Isik(isikukood, riik_kood)
Primaarvõti (isikukood, riik_kood)
Välisvõti (riik_kood) **Viitab** Riik (riik_kood)

Andmebaasis on vaja jõustada reegel, et *riik_kood* peab koosnema alati täpselt kolmes suurtähest A-Z.
(https://en.wikipedia.org/wiki/List_of_ISO_3166_country_codes)

Milliste veergude põhjal tuleks see kitsendus andmebaasis jõustada?

Halb. Ainult tabeli *Isik* veerul *riik_kood*.

Miks see lahendus on halb?

- Tabelis *Riik* võivad olla reeglitele mittevastavad riikide koodid.
- Kui lisandub uus välisvõtme veerg, milles olevad andmed viitavad tabelile *Riik*, siis peab kitsenduse selle veeru korral uuesti jõustama.

Halb. Nii tabeli *Isik* veerul *riik_kood* kui ka tabeli *Riik* veerul *riik_kood*.

Miks see lahendus on halb?

- Kontrolli realisatsioon on dubleeritud. Kui kontrollitav reegel muutub, siis tuleb ka muudatus teha mitmes kohas – rohkem tööd; kui muudatus on osaline, siis tekib süsteemi käitumisse vastuolu.
  - Erandiks on, kui *riik_kood* on mõlemas tabelis defineeritud domeeni põhjal ja kitsendus on seotud domeeniga. Sellisel juhul tuleb kitsenduse muudatus teha ühes kohas (domeenis).

Parem. Ainult tabeli *Riik* veerul *riik_kood*. Tänu välisvõtme mehhanismile on tagatud, et tabeli *Isik* veergu *riik_kood* saab panna ainult selliseid riigi koodi väärtuseid, mis on registreeritud tabelis *Riik*.

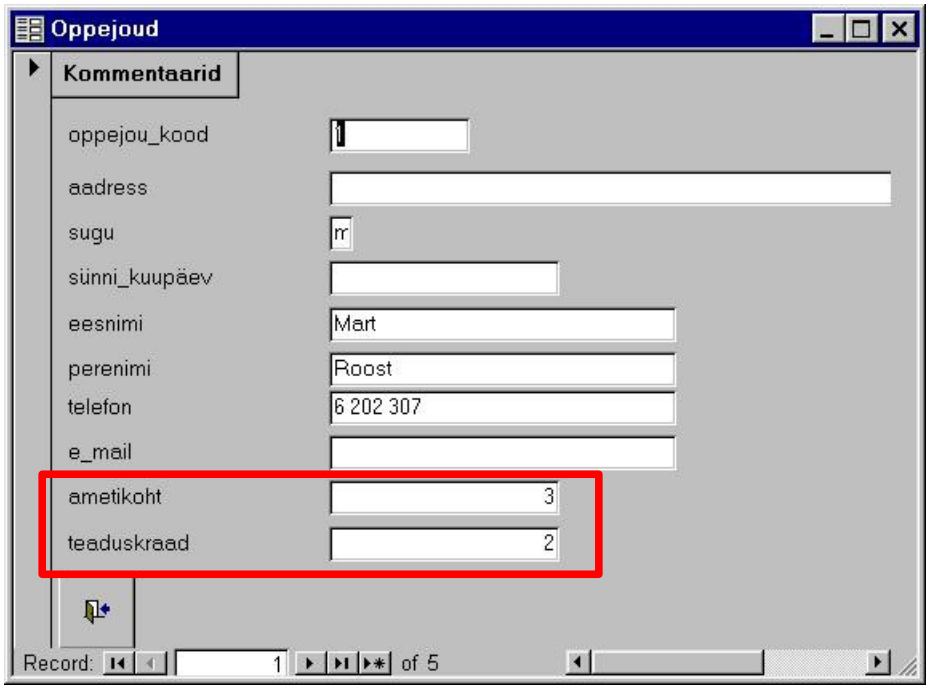
3 Vead kasutajaliideses

Järgnevalt kirjeldatakse tüüpvigu andmebaasirakendustes. Need vead võivad esineda sõltumata rakenduse tüübist (kasutaja arvutis asuv *vs.* veebirakendus) ning rakenduse tegemise vahendist.

3.1Viga 44 (seotud andmeid ei saa valida)

Ekraanivormides peab välisvõtme väärtuste registreerimiseks pidama meeles välisvõtme veerus lubatud väärtuseid.

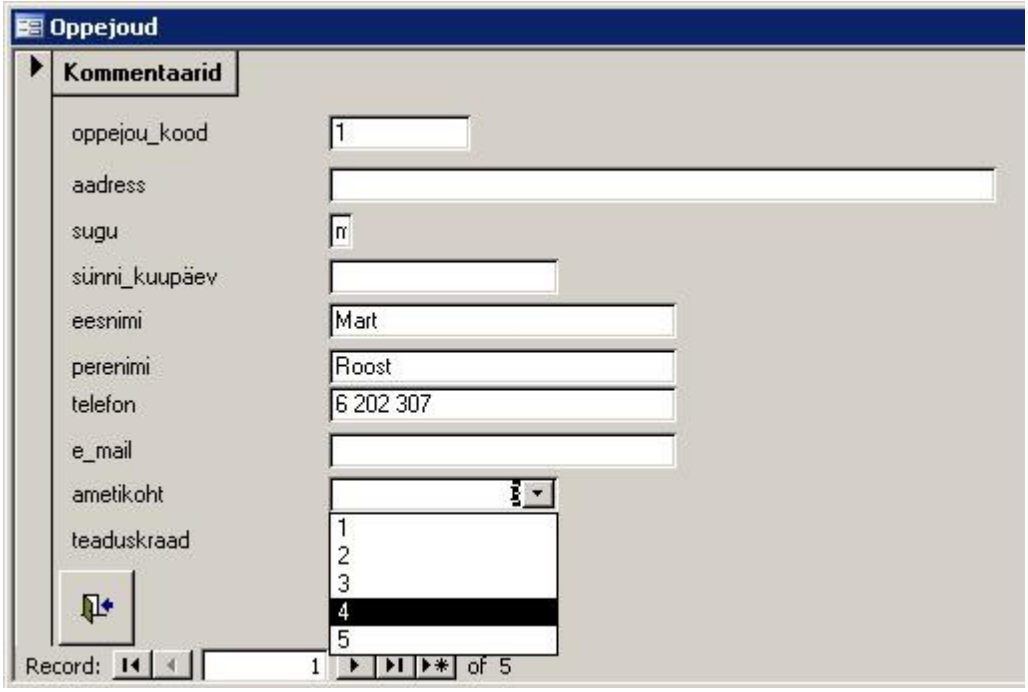
Kommentaar: Tuleb kasutada ripploendeid (*combo box*), hüpikmenüüsid või loendikaste.



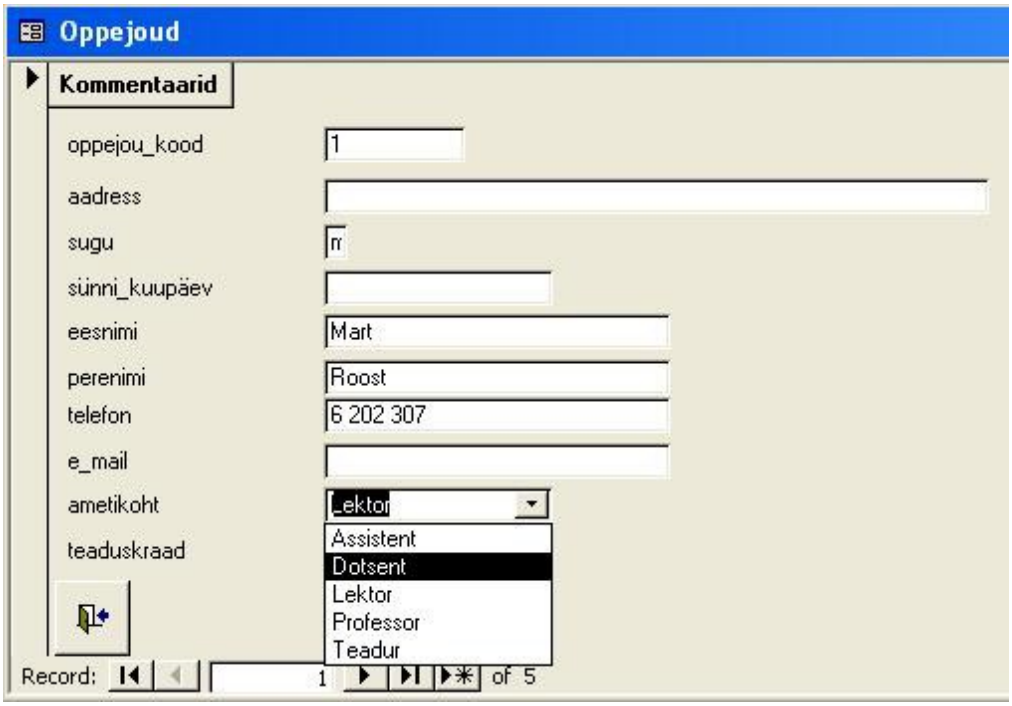
Näiteks ametikoht ja teaduskraad peaks olema võimalik valida nimekirjast. Nimekirjades näidatavad andmed saadakse klassifikaatorite tabelitest.

Rippoendis (või muus kasutajaliidese elemendis) näidatavad andmed peavad vastama mingile tabeli *supervõtmele* – iga valiku rida peab olema andmete poolest unikaalne ja kasutaja jaoks sisuliselt eristatav teistest ridadest.

Halb. Ripploendis (või muus kasutajaliidese elemendis) näidatavad andmed ei ole inimkasutajale piisavalt arusaadavad. Inimkasutajad eristavad ametikohti pigem nimetuste järgi. Praegusel juhul peab kasutaja peast teadma, milline kood tähistab millist ametikohta.



Parem. Ripploendis (või muus kasutajaliidese elemendis) näidatavad andmed on kasutajale paremini arusaadavamad ja tal on lihtsam teha valikut. Antud lahenduse kasutamise eeldus on, et tabelis *Amet* on deklareeritud kitsendus, mille kohaselt veerus *nimetus* olevad andmed peavad olema unikaalsed (ei tohi olla kahte rida, kus on samasugune nimetus).



3.2Viga 45 (olemid pole eristatavad)

Olemite üksteisest eristamiseks ei esitata piisavalt andmeid.

Kommentaar: See probleem võib esineda näiteks nii ripploendites, hüpikakendes, loendikastides kui ka nimekirjavormides.

Halb. Antud näite korral tuleb valida isik, kuid isikute üksteisest eristamiseks ei esitata piisavalt andmeid. Maailmas (ning ka Teie infosüsteemi kasutajate hulgas) võib leiduda mitu sama nimega isikut.
https://www.statista.com/statistics/279713/frequent-combinations-of-first-and-last-name-in-the-us/

Mitmel inimesel võib olla ühesugune nimi. Võib juhtuda, et pole üheselt selge, kes on taotluse esitaja, taotluse sisestaja ja muutja.

põhjus	kommentaar	taotleja	muutja	kellele
Eksam		Galina Rastvortseva	Galina Rastvortseva	Erki Eessaar
Eksam		Galina Rastvortseva	Galina Rastvortseva	Erki Eessaar

Kui on mitu sama nimega õppejõudu, siis pole selge, kes täpselt on ainega seotud.

	ainekood	aine nimetus	õppejõu eesnimi	õppejõu perekonnanimi
	IDU0230	Andmebaasid II	Erki	Eessaar
	IDU0230	Andmebaasid II	Erki	Eessaar

Näide lõputööde teemade haldamissüsteemist *Protsessor*, kus erinevatel aastatel võib õppejõud pakkuda välja sama nimega lõputöö teema, kuid oma teemade nimekirjas ei saa ta aru, millise aastaga on konkreetne teema pakkumine seotud.

Parem. Olemi kohta kasutajale andmeid esitades, tuleb esitada väärtused, mis moodustavad selle **supervõtme**. Supervõti tagab unikaalsuse, kuid selles võib ka olla atribuute, mis pole unikaalsuse tagamiseks vajalikud. Kasutajale tuleb näidata andmeid, mille alusel ta oskab sisuliselt valikut teha. Seega, kui näiteks iga isiku kohta on andmebaasis süsteemi genereeritud unikaalne arv, kuid kasutajatele ei ütle see arv midagi, siis see pole sobilik andmeväärtus, mida kasutajatele näidata.

Järgnevas näites esitatakse isiku valimisel iga isiku kohta tema ees- ja perenimi ning e-posti aadress. Lisaks peaksid valiku read olema sorteeritud, et lihtsustada nende alusel valiku tegemist. Sellise lahenduse kasutamise eeldus on, et iga isiku korral peab olema selline väärtuste kombinatsioon garanteeritult unikaalne ning määratud

(ühegi isiku puhul ei tohi ükski nendest väärtustest puududa) – tegemist on tabeli *Isik* **supervõtmega**. Tabelis peaks olema veerule *e_meil* deklareeritud unikaalsuse ja kohustuslikkuse kitsendused (või siis peab (e_meil) olema selle tabeli primaarvõti).

Järgnevas näites (Tallinna Tehnikaülikooli ÕISi põhjal) kuvatakse lisaks isiku nimele ka tema struktuuriüksust. Nimekirjas on kaks Rein Kuusikut, kes tõsi küll antud juhul töötavad erinevates struktuuriüksustes ja seega on esitatud info piisav nende eristamiseks. Kuid pole mingit garantiid/piirangut, et sama nimega inimesed ei tohiks töötada samas struktuuriüksuses. Sellisel juhul ei pakuks nimekiri piisavalt alust isikute eristamiseks.

Juhendajad Abi

1-9 // 9

	perekonnanimi	eesnimi	struktuuriüksus
Vali	Kauk-Kuusik	Marit	KYP - pooljuhtmaterjalide tehnoloogia õppetool
Vali	Kuusik	Aare	EKE - keskkonnakaitse aluste õppetool
Vali	Kuusik	Alar	IEE - siduselektroonika õppetool
Vali	Kuusik	Argo	EKV - veetehnika õppetool
Vali	Kuusik	Kevin	M - mehaanikateaduskond
Vali	Kuusik	Kristine	H - sotsiaalteaduskond
Vali	Kuusik	Marko	A - energeetikateaduskond
Vali	Kuusik	Rein	KL - anorgaaniliste materjalide teaduslaboratoorium
Vali	Kuusik	Rein	IDN - informaatika aluste õppetool

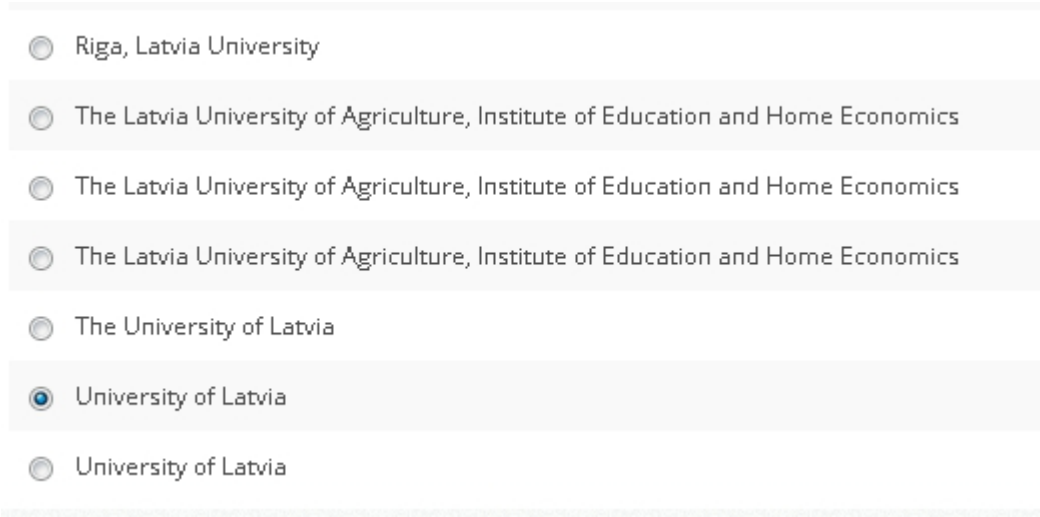
Uus otsing Sulge

Lõputöö teemade näites peaks lisaks teema nimele olema aastaarv või õppeaastale vastava teemade rühma nimi.

3.3Viga 46 (valiku tegemisel korduvad andmed)

Valiku tegemisel esitatakse sama variant korduvalt.

Halb. Lõppkasutajad näevad kasutajaliideses mitut ühesugust valikut. Kasutaja on sellest häiritud ja ei tea, kuidas jätkata.

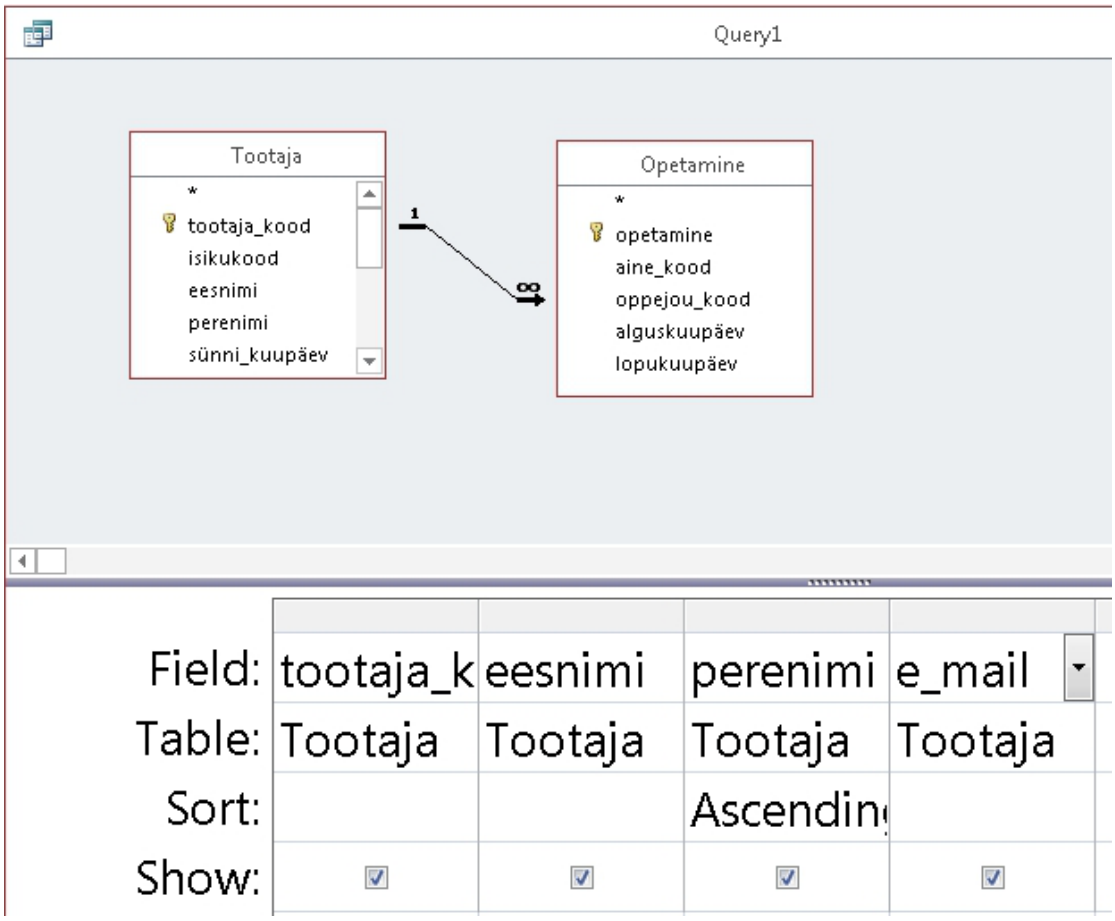


Mis on korduvate variantide esinemise põhjus?

- Olemite eristamiseks ei esitata kasutajaliideses piisavalt andmeid. Andmebaasis registreeritakse andmed kahe või enama erineva olemi kohta, mille kasutajale nähtavad andmed on ühesugused.
- Baastabelites ei ole jõustatud kõiki kandidaatvõtmeid. Seetõttu on sama olemi andmed registreeritud korduvalt.
- Päringud küsivad andmeid üleliigsetest tabelitest.

Kommentaar: Oletame, et iga õppejõud (töötaja) on seotud null või rohkema õpetamisega ning iga õpetamine on seotud täpselt ühe õppejõuga (töötajaga). Järgneva päringu tulemuses on iga töötaja andmed vähemalt üks kord. Kui töötajal

on seotud õpetamisi, siis on tema andmed päringu tulemuses nii mitu korda, kui mitu õpetamist on temaga seotud.



Query1

tootaja_koo	eesnimi	perenimi	e_mail
3	Erki	Eessaar	Erki.Eessaar@ttu.ee
3	Erki	Eessaar	Erki.Eessaar@ttu.ee
3	Erki	Eessaar	Erki.Eessaar@ttu.ee
3	Erki	Eessaar	Erki.Eessaar@ttu.ee
3	Erki	Eessaar	Erki.Eessaar@ttu.ee
3	Erki	Eessaar	Erki.Eessaar@ttu.ee
5	Aavo	Kala	
9	Kalev	Kask	
6	Karl	Kuusk	
1	Mart	Roost	
4	Testkasutaja	Testkasutaja	

Selles päringus ei oleks olnud vaja tabeli *Opetamine* poole pöörduda.

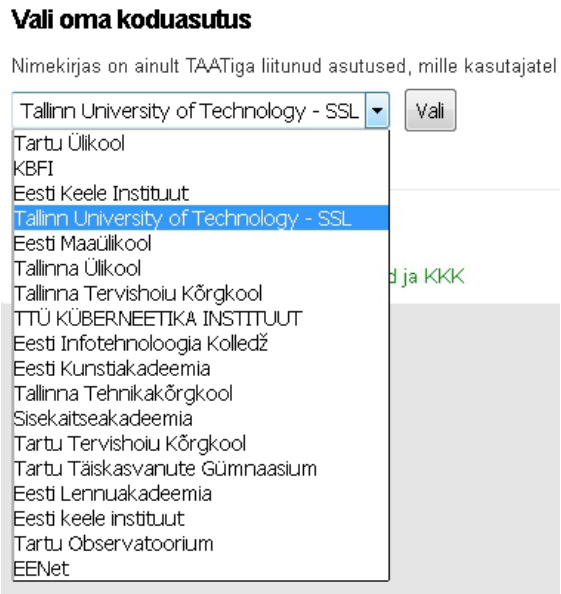
Hea. Valikus on iga variant üks kord. Kasutajale esitatakse piisavalt andmeid, et ta oskaks variantide vahel valida.

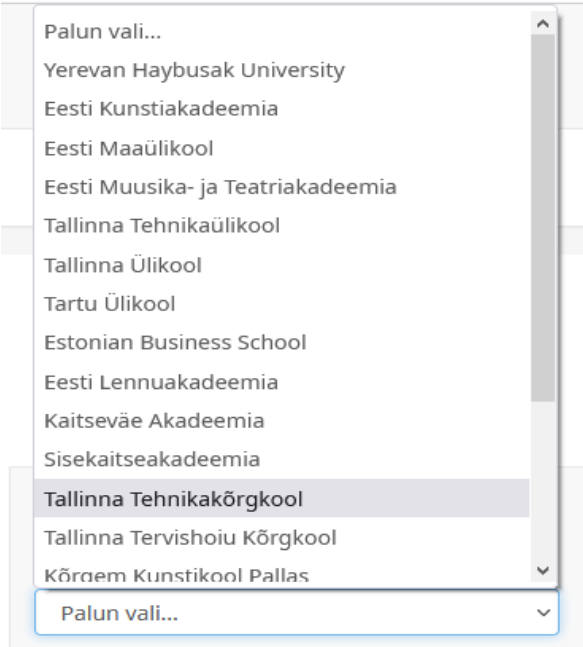
3.4Viga 47 (valiku tegemisel sorteerimata andmed)

Kasutajatele esitatakse valiku tegemisel sorteerimata andmed või on sorteerimiseks kasutatud väärtuseid, mida kasutaja kasutajaliidese kaudu ei näe.

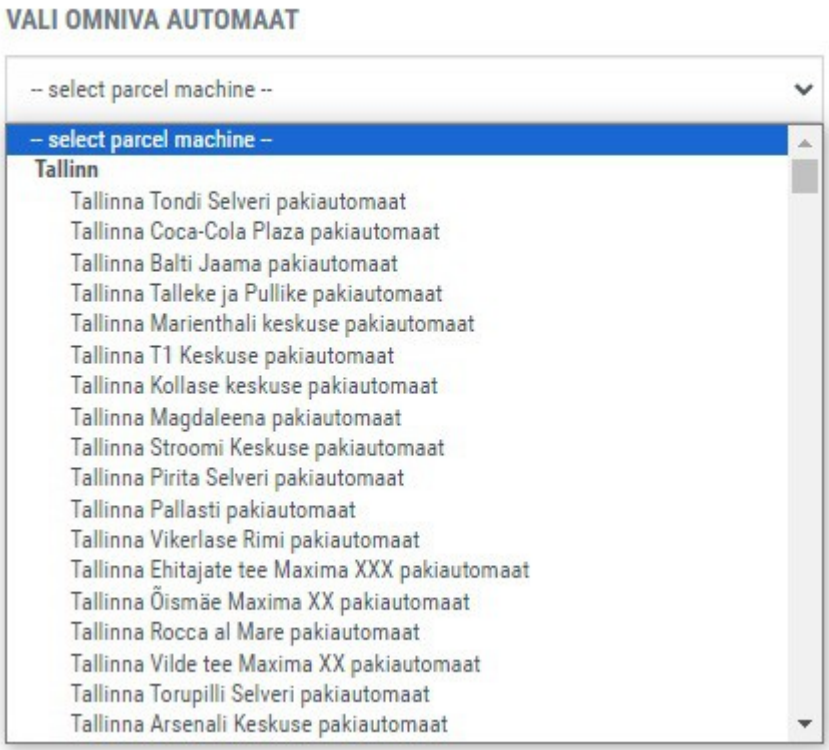
Kommentaar: Sorteerimine on üks lihtsamaid mooduseid andmete otsimiseks.

Halb. Lõppkasutajal kulub otsitava leidmiseks palju aega, sest halvimal juhul peab ta vaatama läbi kogu nimekirja.





PATSIENT
Minu koondvaade
Broneeringu
Broneerige aeg
Eriala:
Oftalmoloogia (silmaarst)
Tervishoiutöötaja:
LÕHMUS, ANN-MARII
DETOTŠENKO, ALEKSEI
Petrova, Tatjana
PUTNIK, MARILIIS
TAMSALU, MARI
Sologubov, Vitali
Rebrov, Margarita
VAHTRAMÄE, AIVE
Tarum, Ann
1-9/15


VALI OMNIVA AUTOMAAT
-- select parcel machine --
-- select parcel machine --
Tallinn
Tallinna Tondi Selveri pakiautomaat
Tallinna Coca-Cola Plaza pakiautomaat
Tallinna Balti Jaama pakiautomaat
Tallinna Talleke ja Pullike pakiautomaat
Tallinna Marienthali keskuse pakiautomaat
Tallinna T1 Keskuse pakiautomaat
Tallinna Kollase keskuse pakiautomaat
Tallinna Magdaleena pakiautomaat
Tallinna Stroomi Keskuse pakiautomaat
Tallinna Pirita Selveri pakiautomaat
Tallinna Pallasti pakiautomaat
Tallinna Vikerlase Rimi pakiautomaat
Tallinna Ehitajate tee Maxima XXX pakiautomaat
Tallinna Õismäe Maxima XX pakiautomaat
Tallinna Rocca al Mare pakiautomaat
Tallinna Vilde tee Maxima XX pakiautomaat
Tallinna Torupilli Selveri pakiautomaat
Tallinna Arsenali Keskuse pakiautomaat

Parem. Isikunimed peaksid olema sorteeritud perenime ja selle sees eesnime järgi tähestikus järjekorras. Asutused peaksid olema sorteeritud nime järgi tähestiku järjekorras. Asukohad peaksid olema sorteeritud aadressi järgi tähestiku järjekorras.

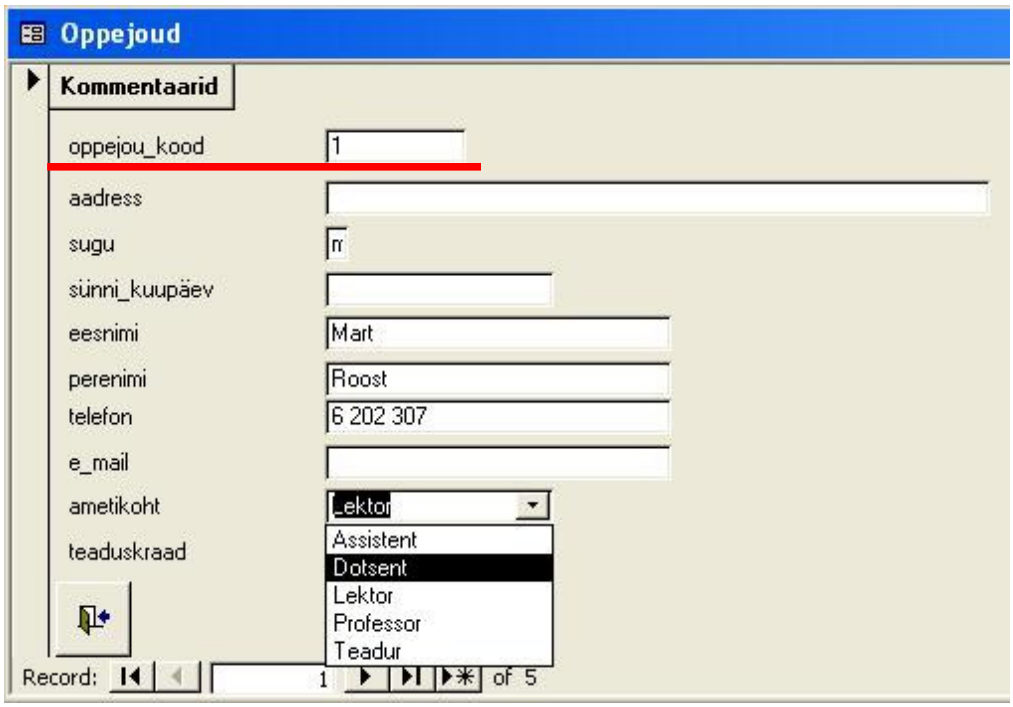
Kuna kasutajaliides on eesti keeles, siis peaks sorteerimine toimuma eesti keele reeglite alusel.

3.5Viga 48 (kasutaja näeb surrogaatvõtmete väärtuseid)

Rakenduses esitatakse kasutajatele surrogaatvõtmete väärtuseid.

Kommentaar: Surrogaatvõtmete väärtuseid kasutatakse, et andmebaasis tabeli ridu üksteisest eristada ning need ei kanna andmebaasi kasutaja jaoks mingit sisulist tähendust. Surrogaatvõtme väärtuseid kasutatakse tarkvaras, kuid neid ei kuvata lõppkasutajale.

Näiteks kui tabelis *Oppejoud* on (oppejou_kood) surrogaatvõti, siis ei peaks selle väärtuseid rakenduses kasutajatele näitama. Kuid kui õppejõudude koode kasutatakse õppejõudude identifitseerimiseks ka väljaspool andmebaasi, siis tuleks neid väärtuseid rakenduses kasutajatele näidata.



Kui rakenduse kasutajatele näidata surrogaatvõtme V väärtuseid, siis võib V aja jooksul muutuda sisulise tähendusega võtmeks, sest kasutajad hakkavad V väärtuseid kasutama ka väljapool andmebaasi.

3.6Viga 49 (väljad ei eristu piisavalt)

Ekraanivormidel ei eristata ainult lugemiseks mõeldud välju lugemiseks ja muutmiseks mõeldud väljadest ning kohustuslikke välju mittekohustuslikest väljadest.

Kommentaar: Erineval viisil/otstarbel kasutatavaid välju tuleks üksteisest visuaalselt eristada. Kasutatavad visuaalsed eristusviisid peavad olema kogu rakenduse ulatuses ühesugused. Kõik see kokku parandab rakenduse kasutatavust.

Halb. See, millistes väljades saab andmeid muuta ja millistes mitte ning millised väljad on kohustuslikud ja millised mitte, selgub kasutajale katse-eksituse meetodil.

Parem. Kohustuslikud väljad on tähistatud tärniga. Väljadel, kus saab andmeid muuta on teistsugune taustavärv (valge) kui väljadel, kus ei saa andmeid muuta (hall).

3.7Viga 50 (ebasobiva suurusega väljad)

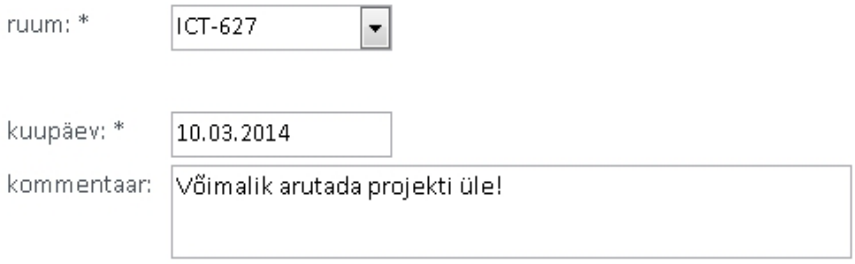
Ekraanivormidel tekstiliste andmete sisestamiseks ja kuvamiseks kasutatavate tekstikastide suurus ei arvesta andmebaasis deklareeritud sellise väärtuse maksimaalse lubatud suurusega.

Näide:

Halb. Kommentaar võib olla kuni 255 märki, kuid selle vaatamiseks/muutmiseks mõeldud väli on sellega võrreldes nagu lukuauk.

Näide Moodle keskkonna sisselogimise aknast, kus kasutajanime (e-meili), sisestamise väli on selgelt liiga väike.

Parem. Kommentaari väli kasutajaliideses arvestab kommentaari väärtuse maksimaalse suurusega.



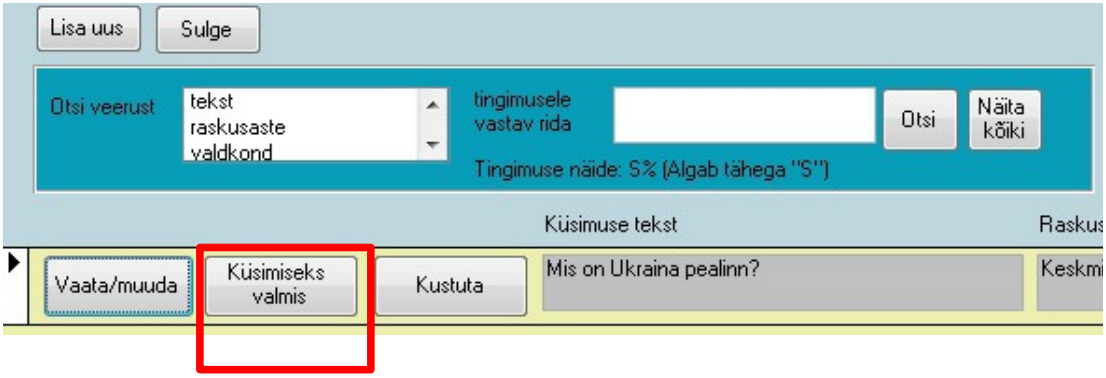
3.8Viga 51 (seisundi valimine)

Seisundimuudatuste registreerimiseks tuleb seisund valida ripploendist või hüpikaknast kõigi võimalike seisundite seast.

Näide:

Halb. Süsteem ei anna kasutajale infot, millistesse seisunditesse üleminek on konkreetse küsimuse puhul lubatud ja millistesse mitte ning kasutaja peab töötama katse-eksituse meetodil. *Eriti halb* on lahendus siis, kui puudub igasugune rakenduse või andmebaasisüsteemi poolne kontroll kas tehtud seisundimuudatus on lubatav (kooskõlas seisundidiagrammiga) või mitte.

Parem. Konkreetses nimekirjavormis kuvatakse vaid seisundis "Koostamisel" olevaid küsimusi. Kuna seisundis "koostamisel" olevaid küsimusi saab viia seisundisse "Küsimiseks valmis", kuid mitte seisundisse "Mitteaktuaalne", siis on nimekirjavormil ekraaninupp "Küsimiseks valmis", kuid pole ekraaninuppu "Mitteaktuaalne".



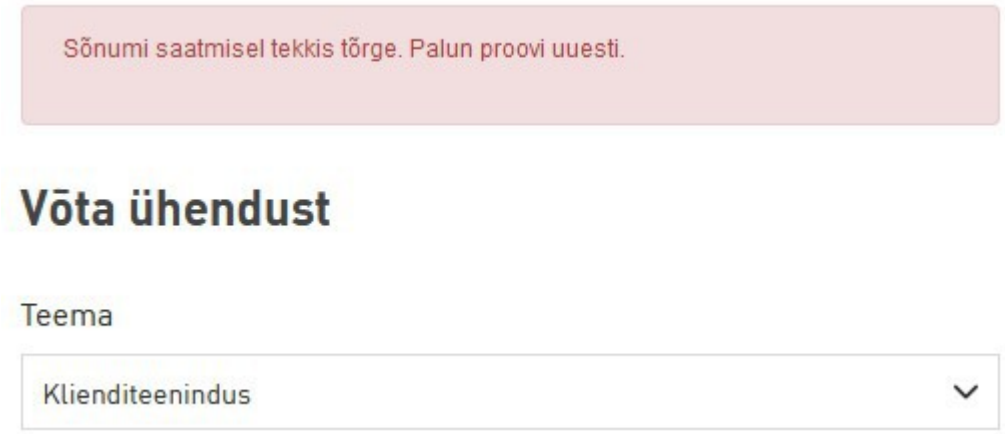
3.9Viga 52 (halvad veateated)

Liiga üldised, eksitavad või puhta valed veateated.

Näide:

Halb. Veateade on täiesti vale ja probleem polnud selles, mida veateate tekst väitis.

Näide: Veateade peale pika sõnumi kirjutamist.



Halb. Ebaselge, milles oli probleem. Kas sõnum oli liiga pikk, selles oli ebasobivaid märke või sõnu või oli põhjus milleski muus?

Veateadete küsimusi uuriti bakalaureusetöös:

Põlluste, K.K., 2015. *SQL lausete veateadete headuse analüüs kolme erineva andmebaasisüsteemi näitel*. Bakalaureusetöö. TTÜ Informaatikainstituut. [WWW] https://digi.lib.ttu.ee/i/?3455 (18.01.2026)

See, et veateated on arusaamatud ja ebaselged on probleem väga paljudes tarkvarasüsteemides. Sellele juhiti tähelepanu juba 1970-ndate ja 1980-ndate aastate uurimistöödes, kuid ka praegu on probleemid samad. Põlluste (2015) kommenteerib, et viga on erand ja tüütus, sest see häirib programmi elegantset tööd. Kui programm võetakse kasutusele ning selle veateated on lootusetud, siis

selle järgnevad mitmed parandused, kuni kasutajad hakkavad vähem kaebama. Sellised süsteemid ei saa kunagi kasutajasõbralikeks.

Seda ei peaks loomulikult võtma, kui vabandust, mis lubab enda süsteeme halvasti disainida.

3.10Viga 53 (ebajärjekindel formaatimine)

Rakenduse erinevatel lehekülgedel/vormides on samad andmed erinevas formaadis.

Kommentaar: Ühel ja samal väärtusel võib olla mitu kasutajatele mõeldud esitusviisi. Näiteks kuupäevastring 04.03.2016 võib tähendada ühes maailma piirkonnas **4. märts 2016** ja teises **3. aprill 2016**. Vaadake kuupäevade esitusviise riikide kaupa: https://en.wikipedia.org/wiki/Date_format_by_country

Kui sama rakenduse sees tähendab väliselt ühesugune esitusviis tegelikult erinevaid väärtuseid, siis on segadus garanteeritud. Õpitavust ja arusaadavust vähendab ka see, kui sama väärtus esitusviis on rakenduse erinevates osades erinev.

Arvestades sellega, et rakendust võivad kasutada erineva kultuuritaustaga isikud oleks mõistlik pakkuda kuupäeva/ajatempli registreerimiseks kalendrivalikut ning esitada välja juures abiinfo selle kohta, kuidas tuleb väärtust interpreteerida.

3.11Viga 54 (ebaselge andmete tähendus)

Kasutajaliideses pole võimalik aru saada andmete tähendusest.

Kommentaar: Väljadest andmete lugeja või andmete muutja peab saama aru, mida talle näidatavad väärtused täpselt tähendavad – milline on nende interpretatsioon. Vastasel juhul võib ta langetada valesid otsuseid. Kasutajaliidese looja saab selleks abi kontseptuaalse andmemudeli osaks olevast olemitüüpide ja atribuutide definitsioonidest.

Halb. Ekraanivorm ei too välja, milliseid mõõtühikuid kasutatakse pikkuse, laiuse ja koguse mõõtmiseks, milline on hinna valuuta ning mida hind ja kogus täpsemalt tähendavad.

pikkus	laius	kõrgus	hind	kogus
150	50	200	22,1	17
153	74	211	330	10

Kuskil pole kirjas, et kasutajalt soovitakse mobiiltelefoni numbrit. Lisaks, sõltuvalt tellimuse tegemise viisist, pole ettevõttel mõnikord üldse vaja telefoni teada.

Parem. Väljadega seotud lipikud nimetavad mõõtühikuid ja väljade täpsemat tähendust.

pikkus (cm)	laius (cm)	kõrgus (cm)	müügihind (EUR)	kogus laos (tk)
150	50	200	22,1	17
153	74	211	330	10

Mõõtühikut või valuutat võib soovi korral näidata ka koos väärtusega. Kui erinevate olemite korral võib atribuudi väärtus olla seotud erineva mõõtühiku või valuutaga, siis see on ainuvõimalik lahendus.

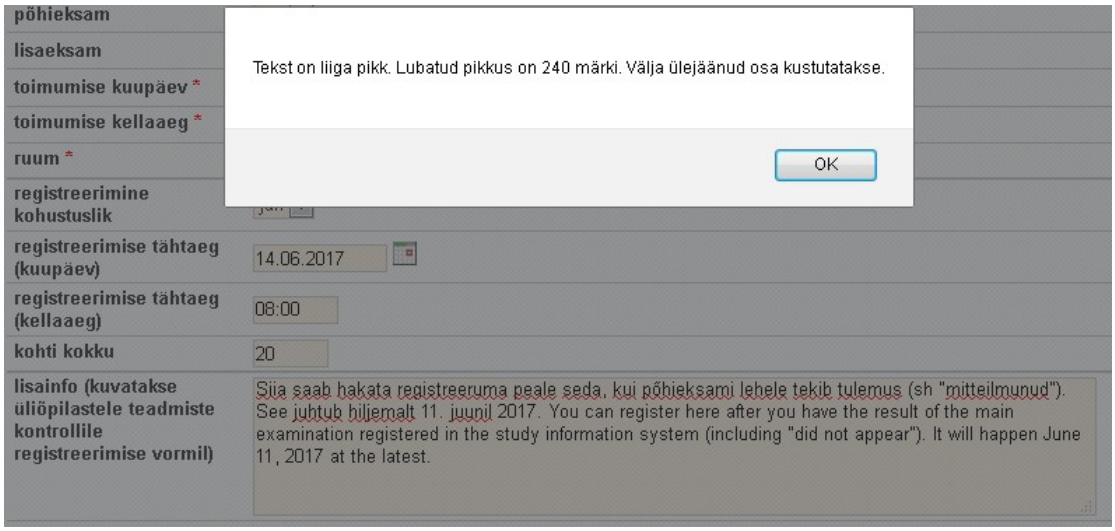
pikkus (cm)	laius (cm)	kõrgus (cm)	müügihind	kogus laos (tk)
150	50	200	22,10 €	17
153	74	211	330,00 €	10

3.12Viga 55 (ebapiisav info piirangute kohta)

Kasutaja saab sisestatavatele andmetele mõeldud piirangutest teada katse-eksituse meetodil. Osa salvestatavatest andmetest läheb kaotsi.

Kommentaar: Kitsendused on head ja vajalikud, kuid need ei tohiks olla liiga piiravad. Süsteem peaks kasutaja tehtud tööd austama, mitte selle "maha lõikama".

Halb. Andmete salvestamisel saan teada, et tekst ei tohi olla pikem kui 240 märki ning sellest pikem osa läheb kaotsi.



Parem. Järgnevaid parandusi oleks kõige parem kasutada üheskoos.

- Väljas lubatud teksti pikkus on suurem.
- Väljaga seotud lipik annab infot lubatud märkide arvu kohta.
- Vea tekkimisel andmeid ei salvestata ning välja kirjutatud tekst säilitatakse, et andmete haldaja saaks sellega edasi tegeleda.

4 Soovitused

Selles peatükis esitatakse mõned üldised soovitused infosüsteemide kavandajatele.

4.1Soovitus 1 (äri tundmaõppimine)

Praktika näitab, et paljudel üliõpilastel on teemaks kaupade tarnimine, arvele võtmine, müümine, laoarvestuse pidamine, arvete koostamine, hinnakirjade koostamine jne. Nende tööde sagedaseks probleemiks on liigne lihtsustamine ja äriprotsessidest mitte arusaamine.

Kommentaar: Eelnevalt loetletud funktsionaalsused on tegelikult kõik ka majandustarkvara ülesanded. Vaatamata raamatu vanusele soovitan võimalusel tutvuda raamatuga:

Koov, H. Majandustarkvara. Tallinn, 2001. 152 lk.

Selles raamatus on toodud näiteid erinevate majandustarkvara programmide kohta. Raamat sisaldab nii kasutajaliidese pilte, kui ka funktsioonide ja tausta kirjeldust. Raamatus on juttu nii universaalsetest programmidest, kui ka kindla eluvaldkonna jaoks mõeldud eriprogrammidest.

Universaalsete programmid.

- Raamatupidamine.
- Kliendihaldus.
- Personal.
- Palk.
- Dokumendihaldus.

Eriprogrammid.

- Kauplus.
- Hulgiladu.
- Hotell.
- Transpordi korraldamine.
- Üüriarvestus.
- Muuseum.
- ...

Mis kasu selle raamatuga tutvumine annab?

- Saate aimu, kuidas majandusarvestusega seotud äriprotsessid toimuvad.
- Leiate võimalikke olemitüüpe, atribuute ja seosetüüpe, mille kohta käivaid andmeid tuleb andmebaasis talletada.
- Näete, kuidas kujundada kasutajaliidest.
- Raamatuga tutvudes leiate viited konkreetsetele tarkvaraproduktidele, mille koduleheküljelt võite leida täiendavat informatsiooni.

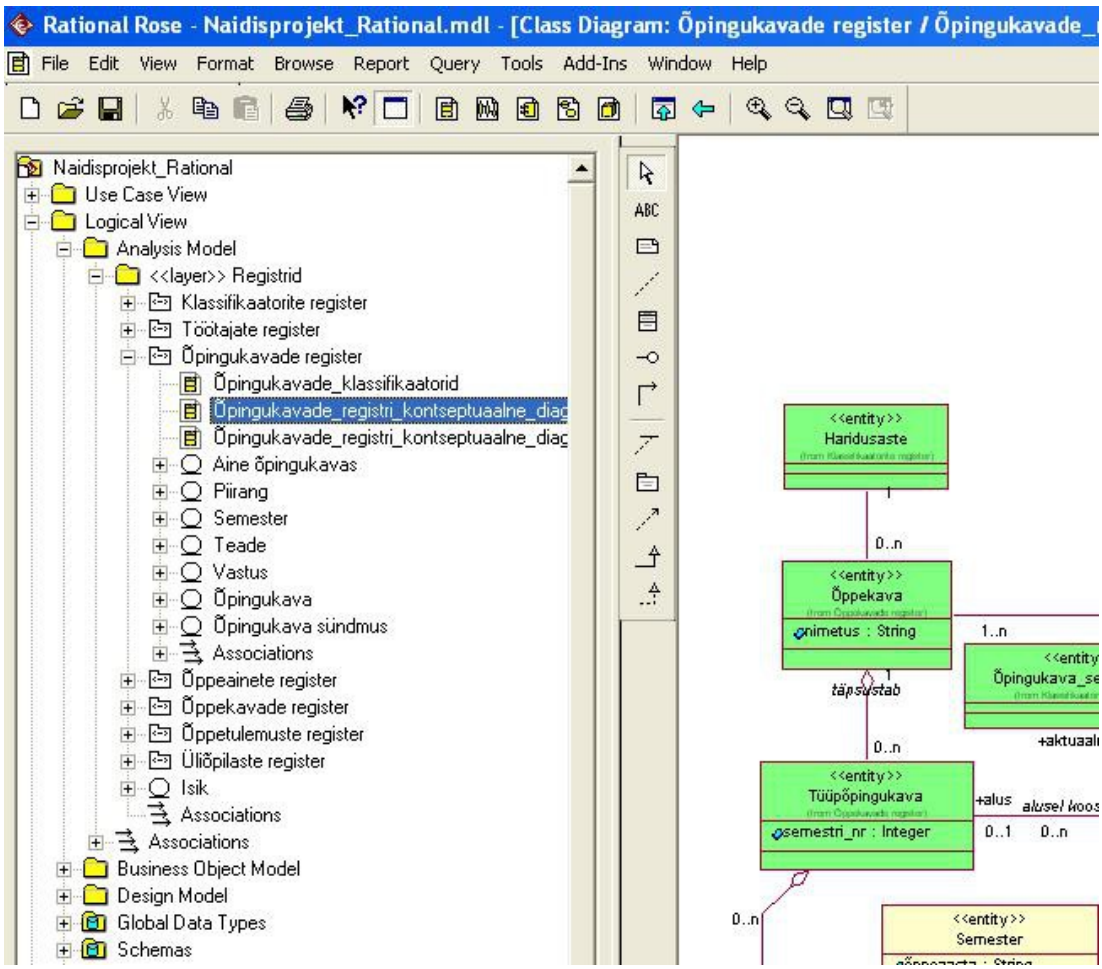
Käesolevas kursuses peate Te andmebaasi ja seda kasutava tarkvara harjutamise mõttes ise otsast peale valmis tegema. Reaalses tööelus aga tuleb süsteemide nullist loomist harva ette. Pigem on vaja olemasolevaid süsteeme täiendada või siis kohandada valmistarkvara (näiteks tarkvara, mida kirjeldatakse majandustarkvara lehel) konkreetse infosüsteemi vajadustele.

4.2Soovitus 2 (pakettide kasutamine)

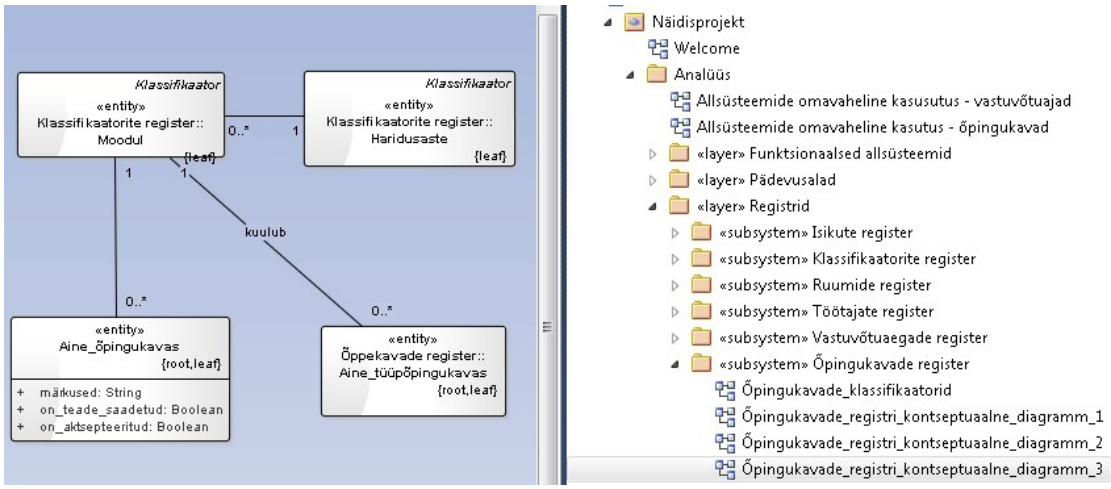
Kasutage pakette, et muuda olemi-suhte diagrammid paremini arusaadavaks.

Kommentaar: Pakett on UMLi mudelielement, mis on mõeldud teiste mudelielementide grupeerimiseks. Iga mudelielemendi omanikuks saab olla ainult üks pakett.

Rationalis modelleerides võiks iga allsüsteemi jaoks luua eraldi paketi. Registrite kirjeldamisel võiks paigutada iga klassi just sellesse paketti, millele vastavat registrit see kirjeldab. Kui nüüd näiteks luua Rationalis diagramm, kuhu kõik või osa nendest klassidest paigutada, siis kuvab Rational klassi sümboli päises paketi (antud juhul siis registri) nime, millesse see klass kuulub. Süsteem ei kuva päises paketi nime ainult sellistel klassidel, mis kuuluvad diagrammiga samasse paketti.



Samasugune funktsionaalsus on ka Enterprise Architect programmis.


Klassifikaator
«entity»
Klassifikaatorite register::
Moodul
{leaf}
Klassifikaator
«entity»
Klassifikaatorite register::
Haridusaste
{leaf}
0..*
1
1
1
kuulub
0..*
0..*
«entity»
Aine_õpingukavas
{root,leaf}
+ märkused: String
+ on_teade_saadetud: Boolean
+ on_aktsepteeritud: Boolean
«entity»
Õppekavade register::
Aine_tüüpõpingukavas
{root,leaf}
Näidisprojekt
Welcome
Analüüs
Allsüsteemide omavaheline kasusutus - vastuvõtuajad
Allsüsteemide omavaheline kasutus - õpingukavad
«layer» Funktsionaalsed allsüsteemid
«layer» Pädevusalad
«layer» Registrid
«subsystem» Isikute register
«subsystem» Klassifikaatorite register
«subsystem» Ruumide register
«subsystem» Töötajate register
«subsystem» Vastuvõtuaegade register
«subsystem» Õpingukavade register
Õpingukavade_klassifikaatorid
Õpingukavade_registri_kontseptuaalne_diagramm_1
Õpingukavade_registri_kontseptuaalne_diagramm_2
Õpingukavade_registri_kontseptuaalne_diagramm_3

Kasutatud materjalid

Veel üks tüüpiline viga seisneb selles, et kasutatud materjale ei panda kirja.

- Date format by country. Wikipedia [WWW] https://en.wikipedia.org/wiki/Date_format_by_country (18.01.2026)
- "Eesti Vabariigi riigiettevõtte põhimääruse" kinnitamise kohta". Elektrooniline Riigi Teataja. [WWW] https://riigiteataja.ee/akt/24040 (18.01.2026)
- Eesti Statistika. Klassifikaatorite portaal. [WWW] https://klassifikaatorid.stat.ee/?siteLanguage=ee
- Isikuandmete kaitse seadus. Elektrooniline Riigi Teataja. [WWW] https://www.riigiteataja.ee/akt/IKS (18.01.2026)
- Isikuandmete töötleja üldjuhend. Andmekaitse Inspektsioon. Tallinn, 2018. [WWW] https://www.aki.ee/sites/default/files/dokumendid/isikuandmete_tootleja_uldjuhend.pdf (18.01.2026)
- List of ISO 3166 country codes. Wikipedia. [WWW] https://en.wikipedia.org/wiki/List_of_ISO_3166_country_codes (18.01.2026)
- ISO 4217 Currency Codes. [WWW] https://www.xe.com/currency/ (18.01.2026)
- Koov, H., 2001. *Majandustarkvara*. Tallinn. 152 lk.
- Larman, C., 2002. *Applying UML and patterns. An introduction to Object-Oriented Analysis and Design and The Unified Process. Second Edition*. Prentice Hall, 627 p.
- List of ISO 639-1 codes. Wikipedia. [WWW] https://en.wikipedia.org/wiki/List_of_ISO_639-1_codes (18.01.2026)
- Põlluste, K.K., 2015. *SQL lausete veateadete headuse analüüs kolme erineva andmebaasisüsteemi näitel.* Bakalaureusetöö. TTÜ Informaatikainstituut. [WWW] https://digi.lib.ttu.ee/i/?3455 (18.01.2026)
- Vellemaa, A.S., 2015. *Mõned disainimustrid klassifikaatorite esitamiseks SQL andmebaasides*. Bakalauruestöö. TTÜ Informaatikainstituut [WWW] https://digi.lib.ttu.ee/i/?3484 (18.01.2026)
- Rahvaraamatukogu seadus. Elektrooniline Riigi Teataja. [WWW] https://www.riigiteataja.ee/akt/12798044?leiaKehtiv (18.01.2026)